Miguel Euripedes Nogueira do Amaral

Desenvolvimento de uma Solução Baseada em *Blockchain* para Armazenamento e Rastreamento de Dados Veiculares

Natal – RN

Janeiro de 2025

Miguel Euripedes Nogueira do Amaral

Desenvolvimento de uma Solução Baseada em *Blockchain* para Armazenamento e Rastreamento de Dados Veiculares

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação.

Orientador: Prof. Dr. Ivanovitch Medeiros Dantas da Silva

Universidade Federal do Rio Grande do Norte – UFRN

Departamento de Engenharia de Computação e Automação – DCA

Curso de Engenharia de Computação

Natal – RN

Janeiro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Amaral, Miguel Euripedes Nogueira do.
Desenvolvimento de uma Solução Baseada em Blockchain para Armazenamento e Rastreamento de Dados Veiculares / Miguel Euripedes Nogueira do Amaral. - 2025.
67f.: il.

Monografia (Graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Engenharia de Computação, Natal, 2025.
Orientação: Prof. Dr. Ivanovitch Medeiros Dantas da Silva.

1. Blockchain - Monografia. 2. Internet dos Veículos - Monografia. 3. Hyperledger Besu - Monografia. 4. Internet das Coisas - Monografia. 5. Rastreamento de Dados - Monografia. I. Silva, Ivanovitch Medeiros Dantas da. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

Miguel Euripedes Nogueira do Amaral

Desenvolvimento de uma Solução Baseada em *Blockchain* para Armazenamento e Rastreamento de Dados Veiculares

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação.

Orientador: Prof. Dr. Ivanovitch Medeiros Dantas da Silva

Trabalho aprovado. Natal – RN, 17 de Janeiro de 2025:

Prof. Dr. Ivanovitch Medeiros Dantas da Silva - Orientador
UFRN

Dr. Breno Santana Santos
UFS

Prof. Dra. Marianne Batista Diniz da Silva
UFAL

Natal – RN

Janeiro de 2025

Dedico este trabalho à minha família, em especial aos meus pais e avós, por toda paciência, incentivo e apoio inestimável ao longo desta jornada.

AGRADECIMENTOS

Agradeço a Deus pelo dom da vida, por todas as oportunidades, pela força e sabedoria concedidas, e pela graça de compartilhar esta jornada com pessoas incríveis, cujo apoio e ensinamentos foram fundamentais para cada conquista.

À minha mãe e maior inspiração, Maria Anarilda Nogueira Lima, cuja dedicação, amor incondicional e sacrifícios tornaram possível cada passo da minha caminhada. Seu apoio incansável, força e coragem foram cruciais para que eu chegasse até aqui, e sou imensamente grato por tudo que fez por mim.

Ao meu pai, Carlos Euripedes Andrade do Amaral, agradeço pelo carinho, pelo incentivo e pelo apoio em minha trajetória, contribuindo para que essa conquista se tornasse realidade.

À minha avó, Maria Zenilda de Lima Nogueira, por todo amor, dedicação e ensinamentos que moldaram quem sou.

À minha noiva, companheira de vida e melhor amiga, Ana Luisa Fernandes de Souza Carvalho, por todo o amor, apoio e cumplicidade em cada passo desta jornada. Sua presença constante torna os desafios mais leves e as conquistas mais especiais.

Aos meus irmãos de vida, Luis Gustavo e Fellipe Milomem, pelo companheirismo, amizade e por tornarem essa jornada mais leve com momentos de descontração e boas lembranças.

Ao meu orientador, professor Ivanovitch Medeiros Dantas da Silva, pela atenção, paciência e incentivo ao longo desta jornada. Pelas oportunidades proporcionadas, pelos ensinamentos compartilhados e, sobretudo, por motivar-me a superar desafios e a evoluir constantemente.

A Marianne Diniz e Breno Santos, pelas orientações generosas, pelo compartilhamento de conhecimento e experiências que contribuíram positivamente para o meu desenvolvimento acadêmico e pessoal.

Aos meus professores, em especial, Luiz Affonso Henderson Guedes de Oliveria e Carlos Manuel Dias Viegas por toda ajuda e ensinamentos passados.

Aos meus amigos do grupo de pesquisa Conect2ai, em especial Matheus Diniz, Morsinaldo Medeiros, Thaís Medeiros, por todo o apoio, colaboração e momentos de aprendizado compartilhados, cuja amizade e contribuições foram essenciais para meu desenvolvimento acadêmico e pessoal.

A todos que, de forma direta ou indireta, contribuíram com a minha formação.

"Every adversity, every failure, every heartache, carries with it the seed of an equal or greater benefit."

(Napolean Hill)

RESUMO

O crescimento da digitalização e da conectividade no setor automotivo tem possibilitado novos modelos de serviços baseados em dados, ampliando a eficiência e a automação dos veículos modernos. Nesse sentido, a crescente convergência entre a Internet das Coisas (IoT) e a IoV, do inglês, *Internet of Vehicles*, tem impulsionado avanços significativos no setor automotivo, destacando a necessidade de soluções tecnológicas para o armazenamento seguro, para confiabilidade e para rastreabilidade de dados veiculares. Neste contexto, a solução proposta apresenta uma arquitetura integrada que utiliza uma rede *blockchain* privada baseada no Hyperledger Besu para armazenar dados de forma segura e imutável. A solução também contempla o desenvolvimento de um sistema intermediário que conecta dispositivos IoT à *blockchain* por meio de uma API, com suporte a uma interface gráfica interativa.O sistema foi validado por meio de uma prova de conceito, na qual foi realizada a coleta de dados veiculares utilizando um dispositivo OBD-II conectado à internet. Os resultados demonstraram que a solução é capaz de consolidar e auditar as informações de maneira eficiente, garantindo não apenas a integridade dos dados armazenados, mas também uma comunicação confiável entre os dispositivos IoT e a *blockchain*, indicando que a proposta pode ser integrada a aplicações reais. Por fim, foi possível concluir o potencial que a integração entre *blockchain* e IoV tem para aplicações críticas, encaminhando a solução para potenciais aplicações com rastreamento confiável de dados sensíveis e incluindo oportunidades para gerenciamento transparente de frotas.

Palavras-chaves: *Blockchain*; Internet dos Veículos; Hyperledger Besu; Internet das Coisas; Rastreamento de Dados.

ABSTRACT

The growth of digitalization and connectivity in the automotive sector has enabled new data-driven service models, enhancing the efficiency and automation of modern vehicles. In this context, the increasing convergence between the Internet of Things (IoT) and the Internet of Vehicles (IoV) has driven significant advancements in the automotive industry, highlighting the need for technological solutions to ensure secure storage, reliability, and traceability of vehicular data. The proposed solution presents an integrated architecture that utilizes a private *blockchain* network based on Hyperledger Besu to securely and immutably store data. The solution also includes the development of an intermediary system that connects IoT devices to the *blockchain* through an API, supporting an interactive graphical interface. The system was validated through a proof of concept, in which vehicular data collection was performed using an OBD-II device connected to the internet. The results demonstrated that the solution can efficiently consolidate and audit information, ensuring not only the integrity of the stored data but also reliable communication between IoT devices and the *blockchain*, indicating that the proposed architecture can be integrated into real-world applications. Finally, the study highlights the potential of *blockchain* and IoV integration for critical applications, paving the way for reliable tracking of sensitive data and presenting opportunities for transparent fleet management.

Keywords: Blockchain; *Internet of Vehicles*; Hyperledger Besu; *Internet of Things*; *Data Tracking*.

LISTA DE ILUSTRAÇÕES

Figura 1 – Organização dos blocos em uma *Blockchain*. 19
Figura 2 – Exemplo de funcionamento de uma função *hash*. 20
Figura 3 – Exemplo de funcionamento da Assinatura Digital. 21
Figura 4 – Ciclo representado o processo de desenvolvimento na linguagem Solidity. 28
Figura 5 – Visão geral da abordagem proposta para solução de registro de dados veiculares em blockchain. 37
Figura 6 – Arquitetura de *Containers* criados para Rede *Blockchain* com o Hyperledger Besu *quickstart*. 39
Figura 7 – Fluxo detalhado da aplicação do Hardhat na Solução. 42
Figura 8 – Visão Geral da proposta para da API a ser elaborada. 42
Figura 9 – Detalhamento do segmento da API destinado a viagem realizadas pelos veículos. 43
Figura 10 – Detalhamento do segmento da API destinado a interagir com a *Blockchain*. 45
Figura 11 – Visualização da interface de usuário proposta para a Solução. 47
Figura 12 – Resultados dos testes realizados com o contrato inteligente proposto. . 50
Figura 13 – Arquitetura de *Containers* criados para Rede *Blockchain* com o Hyperledger Besu, após modificações. 50
Figura 14 – Visualização, pelo Docker Desktop, dos containers da rede em execução. 53
Figura 15 – Interface inicial do explorador de blocos. 54
Figura 16 – Configuração para utilização do *framework* Hardhat. 55
Figura 17 – Comando utilizado para o *deploy* do contrato e demonstração do recebimento do contrato por parte da rede através dos *logs* dos blocos da rede. 56
Figura 18 – Informações obtidas no arquivo *contract.json* após o contrato ser compilado. 56
Figura 19 – Transação de *deploy* do contrato visualizada no explorador de blocos. . 57
Figura 20 – Dispositivo Freematics ONE+. 58
Figura 21 – Mapa da viagem realização. 58
Figura 22 – Parte do CSV processado pela API. 59
Figura 23 – Procedimento de geração do resumo na interface. 60
Figura 24 – Viagem registrada no banco de dados. 60
Figura 25 – Procedimento para submissão de viagem do banco para *blockchain*. . . 60
Figura 26 – Transação de registro da viagem na *blockchain* e suas informações. . . . 61
Figura 27 – Rastreamento das informações salvas na *blockchain*. 62

LISTA DE ABREVIATURAS E SIGLAS

IoIV	*Internet of Intelligent Vehicles*
IoT	*Internet of Things*
OBD	*On-Board Diagnostics*
API	*Application Programming Interface*
SHA	*Secure Hash Algorithm*
PoW	*Proof of Work*
PoS	*Proof of Stake*
PoA	*Proof of Authority*
IBFT	*Istanbul Byzantine Fault Tolerance*
QBFT	*Quorum Byzantine Fault Tolerance*
EVM	*Ethereum Virtual Machine*
BFT	*Byzantine Fault Tolerance*
RPC	*Remote Procedure Call*
JSON	*JavaScript Object Notation*
YAML	*Yet Another Markup Language*
CLI	*Command Line Interface*
HTTP	*Hypertext Transfer Protocol*
SQL	*Structured Query Language*
SoC	*System on a Chip*
GNSS	*Global Navigation Satellite System*
CSV	*Comma-Separated Values*
RPM	Rotações por Minuto
CORS	*Cross-Origin Resource Sharing*

SUMÁRIO

1 INTRODUÇÃO . . . 13
1.1 Objetivos . . . 14
1.2 Justificativa . . . 15
1.3 Classificação da Pesquisa . . . 15
1.4 Estrutura do Trabalho . . . 16

2 FUNDAMENTAÇÃO TEÓRICA . . . 18
2.1 *Blockchain* . . . 18
2.1.1 *Hashs* e Assinatura Digital . . . 20
2.1.2 Mecanismos de consenso . . . 21
2.1.3 Contratos Inteligentes . . . 23
2.1.4 *Blockchain* Privada e *Blockchain* Pública . . . 24
2.2 Ethereum . . . 25
2.2.1 Contas . . . 25
2.2.2 Gás . . . 26
2.2.3 Solidity . . . 26
2.3 Tecnologia Relacionadas a Criação da Rede *Blockchain* . . . 28
2.3.1 Hyperledger Besu . . . 28
2.3.2 APIs de Chamada de Procedimento Remoto em Redes *Blockchain* . . . 30
2.3.3 Docker e Docker-compose . . . 30
2.3.4 Hardhat . . . 31
2.4 Tecnologias de Suporte ao Desenvolvimento do Sistema Proposto . . . 32
2.4.1 Fastify . . . 32
2.4.2 React e Vite . . . 32
2.4.3 PostgreSQL . . . 33
2.5 Trabalhos Relacionados . . . 33

3 ABORDAGEM PROPOSTA . . . 36
3.1 Configuração da Rede *Blockchain* com Hyperledger Besu . . . 38
3.2 Idealização do contrato inteligente . . . 40
3.3 Configuração do Hardhat . . . 41
3.4 Elaboração e funcionamento da API . . . 42
3.5 Banco de dados . . . 46
3.6 Interface de interação com usuário . . . 46

4 EXPERIMENTOS E RESULTADOS . . . 49

4.1 Validação do contrato Inteligente 49
4.2 Implementação da Rede 50
4.2.1 Modificações na rede *quickstart* 50
4.2.2 Bloco gênesis da *blockchain* privada 51
4.2.3 Validando o funcionamento da *blockchain* privada 53
4.3 Prova de Conceito 56
4.3.1 Instrumentação 56
4.3.2 Rota Utilizada 57
4.3.3 Validação da Solução 59

5 CONCLUSÃO 63

REFERÊNCIAS 65

1 INTRODUÇÃO

A evolução tecnológica ao longo das últimas décadas transformou profundamente a maneira como interagimos com dispositivos e sistemas no dia a dia. No setor automotivo, o avanço das tecnologias de comunicação e a ascensão da (IoT), do inglês, *Internet of Things*, possibilitaram a interconexão de veículos, criando oportunidades para diagnósticos mais precisos, otimização de desempenho e coleta de dados em tempo real (PHILIP et al., 2019). Essa integração resultou no surgimento da *Internet of Intelligent Vehicles* (IoIV), em que sensores e sistemas embarcados em automóveis permitem não apenas a captura de informações críticas sobre o veículo, mas também a análise e armazenamento de dados de maneira segura e eficiente (ARTHURS et al., 2022).

A tecnologia *On-Board Diagnostic* (OBD) desempenha um papel crucial nesse cenário, padronizando protocolos de diagnóstico e viabilizando a extração de uma gama de dados veiculares e condições mecânicas do veículo (SIGNORETTI et al., 2020). Dispositivos modernos baseados no protocolo OBD-II, como o Freematics ONE+, permitem capturar essas informações em tempo real, fornecendo detalhes sobre o comportamento do veículo, contribuindo significativamente para aplicações como telemetria, gestão de frotas e sistemas preditivos (SIGNORETTI et al., 2020; Ferreira X. et al., 2023).

Sob essa perspectiva, a coleta de dados veiculares destacou-se impulsionando avanços nas tecnologias de Internet dos Veículos (IoV), entretanto a geração de dados veiculares apresenta desafios significativos relacionados ao armazenamento seguro, rastreabilidade e integridade dessas informações. O conceito de IoV íntegra veículos inteligentes com infraestruturas públicas, sensores, nós de computação e outros componentes do ambiente, promovendo um ecossistema de troca de informações em tempo real que visa melhorar a segurança, eficiência e experiência dos usuários no trânsito (MOLLAH et al., 2021). No entanto, para que os sistemas de transporte inteligentes alcancem seus objetivos, é essencial que a plataforma subjacente de troca e armazenamento de informações seja segura, transparente e imutável (HAMMOUD et al., 2020).

Nesse contexto, a tecnologia *blockchain* surge como uma solução promissora para os desafios do IoV. Originalmente desenvolvida para suportar criptomoedas, a *blockchain* evoluiu para atender às demandas de diversos setores, destacando-se por sua estrutura descentralizada, imutabilidade e capacidade de garantir a integridade dos dados (PANDA et al., 2020). Ao ser aplicada no IoV, a *blockchain* torna-se particularmente relevante, permitindo o registro confiável e auditável de informações, conferindo ao sistema propriedades como segurança, transparência, rastreabilidade e automação. Essa integração é especialmente relevante em ambientes de dados participativos, em que a segurança se torna

um diferencial para mitigar o risco de modificações indesejadas. Dessa forma, por meio da *blockchain*, é possível assegurar que as informações trocadas entre veículos e infraestruturas sejam resistentes a adulterações, ao mesmo tempo em que oferecem suporte à tomada de decisão automatizada, quando associada a tecnologias emergentes (FARAJI; AYACHE; BAKKALI, 2024).

Esse cenário motivou o desenvolvimento do presente trabalho, que propôs não apenas explorar o uso de uma rede *blockchain* privada, baseada no Hyperledger Besu, mas também integrar tecnologias emergentes, com objetivo de armazenar de forma segura dados veiculares. Essa abordagem visa criar uma comunicação entre dispositivos IoT e a *blockchain*, buscando, por meio da solução proposta, garantir a integridade e rastreabilidade das informações coletadas.

1.1 Objetivos

O objetivo geral deste trabalho consiste em aumentar a segurança na coleta e armazenamento de dados gerados por veículos conectados, por meio do desenvolvimento e validação de uma solução baseada em *blockchain* que assegure a rastreabilidade e confiabilidade dessas informações. A proposta abrange as etapas de coleta, pré-processamento e armazenamento distribuído, rastreável e seguro. A solução integra a criação de uma rede *blockchain* privada, utilizando o Hyperledger Besu, com um sistema completo que faz a intermediação entre os veículos e a *blockchain*.

Para consolidar esses objetivo, o estudo pode ser elaborado em três objetivos específicos:

1. Implementar uma rede *blockchain* privada baseada no Hyperledger Besu, configurando os componentes necessários, como validadores e contratos inteligentes, para assegurar o armazenamento seguro, rastreável e imutável dos dados veiculares.

2. Desenvolver um sistema para coleta e pré-processamento de dados veiculares, utilizando dispositivos OBD-II e tecnologias IoT, por meio de uma *Application Programming Interface* (API) que centralize a comunicação e o processamento inicial dos dados. Adicionalmente, criar uma interface gráfica para permitir que usuários acompanhem e interajam com os procedimentos de coleta e processamento em tempo real, oferecendo maior transparência e controle sobre o sistema.

3. Validar a solução proposta por meio de uma prova de conceito em um cenário real, avaliando o sistema proposto, a rede e a comunicação entre eles em um contexto prático de coleta e armazenamento de dados de veículos, avaliando aspectos relacionados a sua eficiência e a rastreabilidade das informações.

1.2 Justificativa

A crescente evolução das tecnologias de IoT e *blockchain* tem gerado profundas transformações no setor automotivo, especialmente no que diz respeito à conectividade e à segurança de dados (OKERE; BALYAN, 2024). A proliferação de sensores embarcados em veículos possibilita a coleta de dados em grande escala, fornecendo informações valiosas sobre o desempenho do automóvel, comportamento do motorista e condições de tráfego. Contudo, esse aumento exponencial de dados traz consigo desafios significativos relacionados à integridade, rastreabilidade e segurança das informações coletadas (DWIVEDI et al., 2020). É nesse contexto que a integração de tecnologias descentralizadas, como a *blockchain*, surge como uma solução promissora para garantir a confiabilidade desses dados em um ambiente seguro e imutável.

A proposta deste trabalho reflete uma necessidade prática de avançar no gerenciamento de dados veiculares, especialmente em um momento em que a digitalização e a interconexão de dispositivos IoT têm se tornado pilares fundamentais da IoV (OKERE; BALYAN, 2024). Ao empregar uma rede *blockchain* privada, a solução apresentada busca resolver desafios específicos de segurança e privacidade, fornecendo um sistema que não apenas armazena os dados de forma distribuída, mas também possibilita rastreabilidade e auditabilidade de todas as informações registradas.

Ademais, a criação de um sistema completo que conecta veículos à *blockchain* é uma característica que alinha o trabalho às demandas reais, proporcionando ao usuário final um controle eficaz sobre suas informações. Essa integração responde diretamente à necessidade de um gerenciamento seguro, permitindo monitorar, registrar e consultar os dados veiculares de forma eficiente. Além disso, o sistema proposto emprega contratos inteligentes para assegurar a imutabilidade das informações e simplificar a automação de processos críticos, como o registro de dados provenientes de veículos. Ambos, o sistema proposto e a rede *blockchain* promovem confiança entre fabricantes, usuários e entidades reguladoras, permitindo aplicações práticas em diferentes níveis do setor automotivo.

Portanto, este trabalho se justifica tanto pela relevância acadêmica quanto pelo impacto prático que oferece. Ao abordar questões críticas de segurança e rastreabilidade, e ao propor uma solução que integra tecnologias emergentes de maneira funcional, a pesquisa contribui para o avanço na aréa e reforça a importância da *blockchain* como ferramenta estratégica para a IoV e outros setores que dependem de dados confiáveis e auditáveis.

1.3 Classificação da Pesquisa

Para atingir os objetivos deste trabalho, uma das etapas fundamentais foi definir a classificação do método de pesquisa. Nesse sentido, a pesquisa adota uma perspectiva

qualitativa, pois o processo centra-se na compreensão da eficácia do sistema proposto em atender aos requisitos de rastreabilidade e confiabilidade dos dados. Desde a obtenção e pré-processamento das informações até o armazenamento na *blockchain*, a análise qualitativa permite avaliar a adequação e o funcionamento integrado da solução desenvolvida.

Com respeito a sua natureza, este trabalho pode ser classificado como aplicado, pois busca gerar soluções práticas para um problema específico: a rastreabilidade e confiabilidade no armazenamento de dados gerados por veículos conectados. Nesse sentido, o foco principal é aplicar conhecimentos teóricos de blockchain e desenvolvimento de sistemas para criar e validar uma solução que possa ser utilizada em cenários reais, visando contribuir fornecendo respostas e aplicações que tenham relação direta com desafios práticos do mundo real.

Quanto aos objetivos, a pesquisa possui um caráter exploratório, dado que investiga a aplicação da tecnologia *blockchain* no armazenamento de dados veiculares, incluindo o desenvolvimento de um sistema intermediário que assegure a eficácia da integração entre os veículos e a *blockchain*. Além disso, são analisadas as possibilidades e limitações da solução proposta, com o intuito de ampliar o conhecimento acerca do tema e identificar aspectos que possam ser aprimorados em futuras implementações.

Por fim, quanto aos métodos, a pesquisa emprega a prova de conceito como procedimento técnico, validando a solução desenvolvida em um cenário prático. A prova de conceito consiste em utilizar um veículo conectado para realizar uma situação de coleta real dos dados, processamento e o armazenamento de informações na *blockchain*, possibilitando a avaliação da solução em condições concretas. Essa abordagem possibilita não apenas validar a aplicabilidade do sistema, mas também identificar melhorias e ajustes necessários para aprimorar seu desempenho e eficácia.

1.4 Estrutura do Trabalho

Este capítulo tem como finalidade introduzir o tema, apresentando o contexto do estudo e as principais motivações para o desenvolvimento da pesquisa. São expostos os objetivos gerais e específicos, bem como a justificativa e a relevância do trabalho. Em seguida, o capítulo 2 aborda a fundamentação teórica necessária para o entendimento e compreensão da solução proposta, discutindo conceitos relacionados a *blockchain* e as tecnologias utilizadas, além disso, são explorados trabalhos correlatos presentes na literatura, objetivando embasar a pesquisa e evidenciar oportunidades que a motivaram. O Capítulo 3, por sua vez, descreve a metodologia empregada no desenvolvimento da solução proposta, detalhando a construção da rede *blockchain*, o sistema de coleta e pré-processamento de dados, bem como as ferramentas e tecnologias utilizadas para a implementação da solução. O Capítulo 4 apresenta os resultados obtidos e as discussões relacionadas à solução proposta, incluindo a realização de uma prova de conceito para

validar sua viabilidade prática. Por fim, o Capítulo 5 conclui o trabalho, destacando as principais contribuições da pesquisa, as limitações enfrentadas e possíveis direcionamentos para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, são apresentados os conhecimentos e tecnologias essenciais para a compreensão da solução de armazenamento de dados veiculares em *blockchain*. O capítulo é estruturado de forma a oferecer uma visão abrangente dos temas necessários ao entendimento do sistema proposto. Primeiramente, são introduzidos os conceitos fundamentais da tecnologia *blockchain*, seguidos por uma discussão aprofundada sobre Ethereum, incluindo sua estrutura e ferramentas associadas, como a linguagem Solidity. Posteriormente, aborda-se o conjunto de tecnologias relacionadas à criação de redes *blockchain*, destacando ferramentas como Hyperledger Besu e Docker. Por fim, são apresentadas as tecnologias utilizadas na implementação do sistema, organizadas para refletir o fluxo do desenvolvimento, desde o *backend*, passando pelo *frontend*, até o banco de dados. Essa abordagem busca conectar as bases teóricas aos aspectos práticos do trabalho, oferecendo um contexto fundamental para a solução proposta.

2.1 *Blockchain*

A *blockchain* é comumente aceita como um subconjunto das tecnologias de contabilidade distribuída. Essa distribuição é alcançada por meio da criação de uma rede peer-to-peer onde os nós são computadores. Nessa rede, cada nó processa transações enviadas pelos clientes. Essas transações tornam-se registros de um banco de dados replicado em todos nós. Estes registros são imutáveis e a rede os agrupa em blocos (ZAND; WU; MORRIS, 2021). Sinteticamente, pode-se entender a *blockchain* como uma base de dados descentralizada e distribuída, que funciona como um registro imutável (MODI, 2022).

A descentralização é um dos pilares da *blockchain*, garantindo que seu funcionamento não dependa de um único servidor ou grupo restrito de servidores. Em outras palavras, mesmo que alguns servidores ou nós da rede apresentem falhas, o serviço ou aplicação baseada em *blockchain* continua operando normalmente. Essa resiliência é garantida porque cada nó da rede possui uma cópia completa dos dados e da lógica de execução, de forma que nenhum servidor individual detém controle absoluto sobre as operações (MODI, 2022; WU; ZOU; SONG, 2023; BASHIR, 2023).

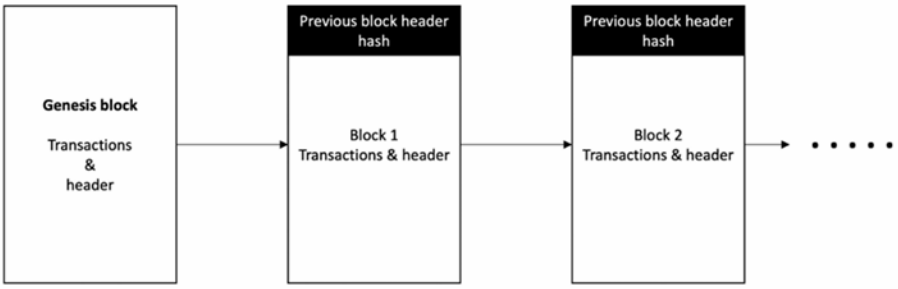
Na *blockchain*, os nós da rede estão conectados de maneira abrangente. Diferentemente de modelos tradicionais que usam conexões ponto-a-ponto (um para um) ou centralizadas (um para muitos), em uma *blockchain* cada nó está conectado a múltiplos outros, formando uma rede muitos-para-muitos. Essa estrutura garante que os dados e as transações sejam replicados de forma eficiente e sincronizada entre todos os participantes

da rede (MODI, 2022).

Assim como em qualquer banco de dados, a *blockchain* permite o armazenamento de informações de forma durável, possibilitando a recuperação dessas informações a qualquer momento. Contudo, sua estrutura vai além de um banco de dados tradicional, pois funciona como um registro especializado em que os dados são armazenados em um padrão apenas de adição (*append-only*), sem possibilidade de modificação direta nas transações já registradas (MODI, 2022; ZAND; WU; MORRIS, 2021).

A *blockchain* é composta por uma sequência de blocos encadeados, como ilustrado na Figura 1. Cada bloco contém um conjunto de transações que são registradas de maneira a impedir alterações posteriores, isso porque no cabeçalho de cada novo bloco temos uma informação (*hash*) que compacta tudo registrado no bloco anterior. Sendo assim, uma vez que um bloco é adicionado à cadeia, suas informações se tornam praticamente imutáveis, devido ao uso de algoritmos criptográficos e outros mecanismos de segurança encadeados. (MODI, 2022; BASHIR, 2023).

Figura 1 – Organização dos blocos em uma *Blockchain*.



Fonte: (BASHIR, 2023).

Esse design confere à rede suas propriedades únicas de transparência, integridade e confiança. Para alterar uma transação previamente registrada, seria necessário criar uma nova transação, mas nunca modificar o histórico já consolidado. Por não permitir alterações ou exclusões, a tecnologia é vista como resistente a fraudes ou corrupção, característica particularmente importante em aplicações onde a integridade das informações é essencial, como sistemas envolvendo contratos, finanças e rastreamento de dados (MODI, 2022).

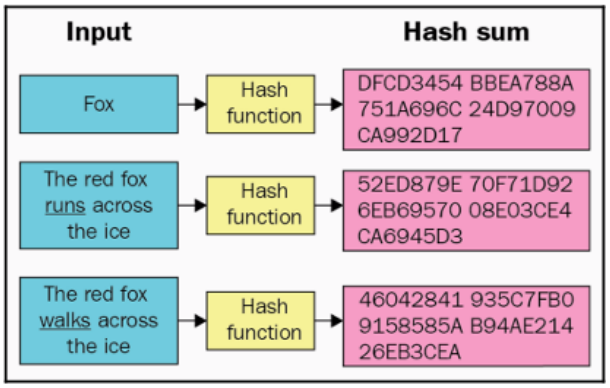
Ao longo dos demais tópicos desta seção, serão discutidos os outros fundamentos da *blockchain*, incluindo os conceitos de *hash* e assinatura digital, que explicam como os dados são protegidos; mecanismos de Consenso, responsáveis por validar as transações na rede; contratos inteligentes, que permitem a execução de lógica programável diretamente na *blockchain*; e a diferença entre *blockchain* pública e *blockchain* privada. Esses tópicos visam proporcionar um entendimento abrangente dos aspectos técnicos e práticos dessa tecnologia.

2.1.1 *Hashs* e Assinatura Digital

A tecnologia *blockchain* combina características como descentralização, distribuição e imutabilidade, sendo sustentada por avançadas técnicas de criptografia, incluindo funções *hash* e assinaturas digitais. Essas ferramentas têm papel fundamental para garantir a segurança e autenticidade na rede.

As principais técnicas de criptografia utilizadas em *blockchain* são a função *hash* e as assinaturas digitais. A função *hash* é uma função matemática unidirecional que não pode ser descriptografada. Ou seja, para qualquer entrada que for fornecida à função, seja uma string, um número ou até um arquivo, independentemente do tamanho, ela irá mapear essa informação para uma sequência de tamanho fixo, como uma *hash* de 128 *bits* ou até mesmo 256 *bits*. Conforme ilustrado na Figura 2, todas as saídas da função *hash*, tem a mesma quantidade de caracteres, independente da entrada usada. Como resultados dessas sequências únicas, temos uma ferramenta resistente a colisões e que é capaz de guardar dados dentro dela (KULKARNI, 2018; LAURENCE, 2019).

Figura 2 – Exemplo de funcionamento de uma função *hash*.



Fonte: (KULKARNI, 2018).

O algoritmo de *hash* segura SHA, do inglês, *Secure Hash Algorithm*, é uma das funções *hash* criptográficas usadas em *blockchains*. SHA-256 é um algoritmo comum aplicado a essas redes, em especial a função de gerar saídas exclusivas de tamanho fixo, 256 *bits* ou 32 *bytes*. Conforme introduzido na seção anterior, na *blockchain*, a *hash* tem o papel de ligar um bloco ao outro, permitindo a ligação de um novo estado da rede com o seu anterior. Sua utilização é fundamental, criando uma impressão digital dos dados registrados no bloco, além de conectar toda a cadeia (LAURENCE, 2019).

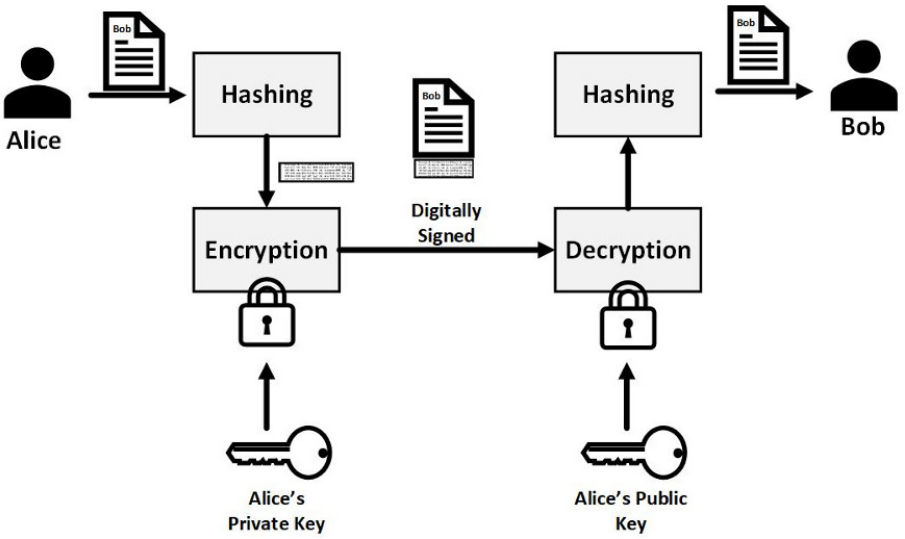
O segundo bloco de criptografia das *blockchains* é o equivalente a uma assinatura escrita a mão. Para isso utiliza-se uma criptografia de chave pública, uma função que permite encriptar e decriptar através do uso de duas chaves diferentes, uma pública e a outra privada. Essas duas chaves são criadas em pares, permitindo que esse tipo de criptografia também seja chamada de criptografia assimétrica. A chave pública pode ser

compartilhada, mas a chave privada deve ser de conhecimento apenas do seu dono. Com essas duas chaves, ambas podem ser utilizadas para encriptar algo, de maneira que a outra seja utilizada para descriptografar (KULKARNI, 2018; WU; ZOU; SONG, 2023).

Dessa forma, destaca-se que na *blockchain*, integra-se os algoritmos de *hash* com as assinaturas digitais, permitindo determinar a autenticidade e integridade de uma mensagem ou documento digital. Através disso, é possível conectar uma mensagem ao seu criador e permitir que essa informação seja assegurada apenas para quem a recebe. Garantindo também que a mensagem não foi alterada desde que foi encriptada com a informação de quem a enviou (WU; ZOU; SONG, 2023).

Como ilustrado na Figura 3, em um cenário hipotético o usuário, Alice, envia uma transação para o usuário Bob. Para isso, a transação passa primeiro por uma função que gera um *hash* das informações e Alice posteriormente utiliza sua chave privada para encriptá-la. Ao receber a transação, o segundo usuário, Bob, pode então acessar as informações da transação utilizando a chave pública de Alice, assegurando privacidade e confiabilidade durante todo o processo.

Figura 3 – Exemplo de funcionamento da Assinatura Digital.



Fonte: (WU; ZOU; SONG, 2023).

2.1.2 Mecanismos de consenso

O consenso é um dos pilares fundamentais das *blockchains*, juntamente com a criptografia e a descentralização. Em sistemas distribuídos de grande escala esse mecanismo permite que os participantes de uma rede descentralizada concordem sobre o estado atual do registro compartilhado, sem depender de uma autoridade central. Isso garante a confiabilidade dos dados registrados na *blockchain*, mesmo em cenários onde os participantes

podem não confiar uns nos outros. Através dos algoritmos de consenso as transações na rede podem ser validadas, ordenadas e adicionadas aos blocos de forma segura, mitigando o risco de fraudes, ataques e inconsistências no sistema. É válido destacar que existem diversos tipos de algoritmos de consenso, cada um com um processo fundamental de validação diferente. Dessa forma, redes *blockchain* distintas podem aplicar mecanismos diferentes e mais apropriados para sua arquitetura e aplicação (WU; ZOU; SONG, 2023; GARG, 2023).

Um dos mecanismos de consenso mais conhecidos é o *Proof of Work* (PoW), utilizado por algumas *blockchains* públicas, como Bitcoin (NAKAMOTO, 2008). No PoW, os participantes, conhecidos como mineradores, competem para resolver problemas matemáticos associado a *hash* de cabeçalho do bloco a ser adicionado na rede. O primeiro a resolver o problema propõe um bloco para validação e recebe uma recompensa por seu esforço. O PoW oferece segurança e resistência a ataques associados a redes descentralizadas (GARG, 2023).

Na proposta inicial, a rede Ethereum utilizava o mecanismo PoW. No entanto, em atualizações recentes, migrou para o *Proof of Stake* (PoS), uma alternativa projetada para superar as limitações do PoW, especialmente seu alto consumo de energia (WOOD, 2024). No PoS, a validação de transações e a criação de novos blocos são realizadas por validadores, que precisam comprometer uma quantidade específica de algo como garantia, na rede Ethereum pode-se usar *tokens*, por exemplo. Essa "aposta"funciona como um incentivo para que os validadores ajam de maneira honesta, uma vez que comportamentos maliciosos podem levar à perda parcial ou total do que foi comprometido. A seleção do validador para propor o próximo bloco é geralmente baseada em critérios como a quantidade usada como garantia e o tempo de participação na rede, mas essa forma de escolha pode mudar dependendo das regras da *blockchain*. Esse modelo aumenta a eficiência da rede, ao passo que reduz significativamente o consumo de energia, tornando-o uma alternativa mais sustentável e alinhada às demandas ambientais atuais (GARG, 2023).

Existem também os mecanismos de *Proof of Authority* (PoA), amplamente utilizados em *blockchains* privadas e consorciadas, que possuem um enfoque diferente. No PoA, a autoridade para validar transações e propor blocos é concedida a um número limitado de nós confiáveis, geralmente identificados e aprovados por um consórcio ou entidade central. Dentre os mecanismos de PoA mais conhecidos, destacam-se o *Istanbul Byzantine Fault Tolerance* (IBFT) e o *Quorum Byzantine Fault Tolerance* (QBFT) (WU; ZOU; SONG, 2023).

O IBFT opera garantindo que, mesmo em presença de nós maliciosos, a rede alcance consenso desde que menos de um terço dos nós sejam comprometidos. Já no QBFT, uma evolução do IBFT, participantes aprovados atuam como validadores, de maneira que os validadores trocam turnos para validar os blocos. Além disso, de maneira similar ao IBFT,

uma maioria, equivalente a dois terços ou mais, deve assinar o bloco. Adicionalmente, o mecanismo permite que, caso seja necessário adicionar ou remover validadores, aqueles existentes proponham e votem, necessitando maioria para obter sucesso em qualquer tipo de proposta. Com isso, o QBFT oferece maior eficiência e flexibilidade a redes privadas, permitindo também ajustar os intervalos de tempo entre blocos e melhorando a tolerância a falhas na rede (QUORUM, 2024).

Os mecanismos de consenso PoA são particularmente adequados para aplicações empresariais, como as construídas com Hyperledger Besu ou Quorum, devido à sua baixa latência e alta eficiência (BESU, 2024a; QUORUM, 2024). No entanto, pode-se concluir que o modelo PoA sacrifica parte da descentralização, uma vez que depende da confiança em um conjunto limitado de validadores. Ainda assim, essa característica torna-o ideal para ambientes onde os participantes compartilham interesses comuns e existem requisitos de governança e conformidade (WU; ZOU; SONG, 2023).

Em resumo, independente do mecanismo, o consenso desempenha um papel crucial em moldar o desempenho, a segurança e a aplicabilidade de uma *blockchain*. A escolha do modelo apropriado, seja PoW, PoS ou PoA, depende das demandas específicas do caso de uso, equilibrando fatores como descentralização, eficiência e governança (LAURENCE, 2019; GARG, 2023).

2.1.3 Contratos Inteligentes

Introduzidos na *blockchain*, particularmente por meio do Ethereum, em 2015, os contratos passaram a ser implementados de forma prática, ganhando relevância como ferramentas essenciais em sistemas descentralizados (LAURENCE, 2019; BUTERIN, 2014). Como definido por Modi (2023), esses contratos são programas autônomos e auto-executáveis, implantados em um ambiente virtual de *blockchain*. Sua funcionalidade baseia-se em regras codificadas que automatizam transações entre partes e ações pré-programadas, permitindo o armazenamento e processamento de dados sem a necessidade de intermediários. Diferente dos contratos tradicionais, que são documentos legais executados sob a autoridade de leis e sistemas judiciais, os contratos inteligentes são executados diretamente na rede, proporcionando eficiência, transparência e auditabilidade.

Os contratos inteligentes, como estruturas digitais semelhantes a classes orientadas a objetos, também podem ser compostos ou interagir entre si. Isso possibilita a criação de sistemas complexos, onde um contrato pode invocar outro para implementar lógica distribuída ou armazenar dados adicionais. Com isso, é possível ampliar as possibilidades de aplicação, implementando ecossistemas integrados em diversas áreas, como finanças, logística, saúde e IoT (MODI, 2022; WU; ZOU; SONG, 2023).

No contexto técnico, os contratos podem realizar leituras e inserções de dados, mas

não têm a capacidade de deletar ou modificar informações já registradas, reforçando a imutabilidade inerente a uma plataforma *blockchain*. Dentro de suas aplicações, podemos citar por exemplo a rede Ethereum, onde a interação entre cliente, contrato inteligente e *blockchain* é mediada por APIs que otimizam a comunicação e execução das transações. Ademais, apesar de serem poderosos instrumentos, a eficiência de um contrato depende de um design cuidadoso para evitar redundâncias e maximizar o desempenho (ZAND; WU; MORRIS, 2021).

2.1.4 *Blockchain* Privada e *Blockchain* Pública

As *blockchains* são classificadas principalmente em públicas e privadas, dependendo de como a rede é configurada e gerenciada. Essa diferenciação baseia-se em critérios como governança, acesso, consenso e escalabilidade, características fundamentais para determinar o uso mais apropriado em cenários específicos. Ambas as categorias compartilham os fundamentos da tecnologia *blockchain*, como registros imutáveis e descentralizados, mas apresentam características distintas que atendem a diferentes demandas de mercado e requisitos técnicos, como flexibilizações em privacidade e algoritmos de consenso diferentes (GARG, 2023).

As *blockchains* públicas, como Bitcoin e Ethereum, são completamente abertas e acessíveis a qualquer pessoa com uma conexão à internet. Elas são caracterizadas por sua natureza *permissionless*, ou seja, qualquer usuário pode participar como nó da rede, validar transações ou criar contratos inteligentes (Ethereum) sem a necessidade de uma autoridade central. O consenso em *blockchains* públicas geralmente é alcançado por meio de mecanismos como PoW ou PoS. Esse tipo de rede, pode estar associado a criptomoedas e por isso, buscam mais segurança, resistência à censura e privacidade na rede (LAURENCE, 2019). Dessa forma, esse modelo torna as *blockchains* públicas focadas para aplicações onde a privacidade é fundamental, mas pode trazer desafios, como custos elevados de operação e limitações associadas à escalabilidade (NAKAMOTO, 2008; BUTERIN, 2014; BASHIR, 2023).

Contrário as redes públicas as rede privadas impõem uma associação em que apenas membros com uma adesão válida participem, operando em sua maioria em ambientes restritos (ZAND; WU; MORRIS, 2021). Oferecendo maior controle e governança, essas *blockchains* utilizam um modelo *permissioned*, no qual os nós são pré-selecionados e controlados por uma entidade ou consórcio (MODI, 2022). O propósito das redes privadas depende dos casos de uso específicos dela e dos participantes da rede. Aplicações comuns desta tecnologia incluem: uso comercial e empresarial, uso em conformidade regulatória e aplicações envolvendo segurança e privacidade dos dados (WU; ZOU; SONG, 2023).

Destaca-se que as *blockchains* privadas podem oferecer vantagens aos desenvolve-

dores em termos de personalização, flexibilidade, transações mais rápidas, privacidade aprimorada e redução da carga regulatória. Entretanto, seu cenário de aplicação deve assegurar que todos os participantes sejam conhecidos (BASHIR, 2023). Nesse contexto, à medida que a tecnologia *blockchain* amadureceu, as empresas começaram a adotar a tecnologia para seus próprios casos de uso, adequando-se mais as mencionadas *blockchains* privadas, graças a característica de proteção aos dados confidenciais e a garantia de controle operacional (WU; ZOU; SONG, 2023). Por outro lado, as redes públicas mostram-se ideais para aplicações como criptomoedas e *marketplaces* descentralizados (LAURENCE, 2019).

2.2 Ethereum

Ethereum, como uma plataforma *blockchain* pioneira, amplia os horizontes das aplicações descentralizadas ao introduzir funcionalidades avançadas, como contratos inteligentes. Esta seção explora as particularidades da rede Ethereum, abordando inicialmente suas contas, elementos fundamentais para a interação na *blockchain*. Em seguida, discute-se o conceito de gás, elemento que assegura a execução de transações e contratos inteligentes. Por fim, o tópico final é a linguagem Solidity, sendo ela predominante para o desenvolvimento de contratos inteligentes no ambiente Ethereum. Esta seção busca oferecer uma visão estruturada das bases que tornam do Ethereum, fundamental para compreender funcionalidades inerentes à rede Hyperledger Besu proposta neste trabalho.

2.2.1 Contas

No ecossistema da rede Ethereum as contas são consideradas blocos fundamentais da rede. De forma que a rede busca, essensialmente, guardar as informações das interações entre as contas no seu registro. Para isso, existem dois tipos de contas na rede Ethereum, contas de propriedade externa e contas de propriedade dos contratos inteligentes. Cada conta tem sua propriedade chamada de balanço, que guarda a informação da quantidade de Ether, a criptomoeda da rede Ethereum, que aquela conta possui (MODI, 2022).

As contas de propriedade externa são aquelas pertencentes aos usuários da rede. É importante destacar que as contas na rede Ethereum não são referenciadas por nome, e sim por uma *hash*. Ao criar uma conta, o usuário adquire sua chave pública e sua chave privada. Com isso, Ethereum usa uma quantidade pré definida dos primeiros caracteres da *hash* para representar a identidade da conta (MODI, 2022; WU; ZOU; SONG, 2023).

Contas de contrato por um lado tem um funcionamento similar, uma vez que são identificados por um endereço público e podem conter um balanço de Ether associado a eles. Entretanto, de forma distinta essas contas não possuem uma chave privada e contém código do contrato, que consiste de funções e variáveis de estado (MODI, 2022).

2.2.2 Gás

Contas de propriedade externa são as responsáveis por iniciar as transações, e qualquer transação na rede ethereum precisa de gás para ser executada. Toda atividade na rede que modifica o seu estado de alguma forma precisa de uma taxa para ser executada. Ou seja, gás é o combustível para execução de transações e ações com smart contracts na *Ethereum Virtual Machine* (EVM), que é a plataforma computacional descentralizada por trás do Ethereum, sendo responsável por garantir a execução dos contratos. Para obter gás o usuário precisa comprá-lo com Ether. De maneira geral, em uma rede pública, esse valor pode ser visto como um incentivo para os validadores executarem aquela transação (MODI, 2022; BASHIR, 2023).

Dois pontos importantes a serem considerados com relação ao gás na rede Ethereum seriam, primeiro, o valor do gás pode mudar, dependendo da demanda de transações da rede e, segundo, caso o contrato seja muito complexo computacionalmente, a demanda de gás para sua execução pode se tornar elevada, podendo impactar a usabilidade e viabilidade de um sistema (WU; ZOU; SONG, 2023).

2.2.3 Solidity

Solidity é uma linguagem de programação projetada especificamente para a criação de contratos inteligentes que são executados na EVM. Desenvolvida por Gavin Wood e outros contribuidores do núcleo do Ethereum, sua sintaxe é semelhante a linguagens amplamente utilizadas como C++ e JavaScript. Solidity é uma linguagem orientada a objetos, estaticamente tipada, que oferece suporte a herança, bibliotecas e tipos definidos pelo usuário. Essas características tornam a linguagem promovida para o desenvolvimento de aplicações descentralizadas, permitindo que desenvolvedores implementem lógicas de negócios complexas em contratos inteligentes (WU; ZOU; SONG, 2023).

A execução de contratos criados em Solidity segue um fluxo predefinido. Primeiro, o código é compilado em *bytecode* que pode ser interpretado pela EVM. Durante a compilação, o compilador verifica as regras de sintaxe, e o *bytecode* gerado é sempre consistente, desde que os parâmetros de entrada e a versão do compilador permaneçam os mesmos. Uma vez implantado, o contrato recebe um endereço exclusivo na rede Ethereum, e os usuários autorizados podem interagir com suas funções por meio de transações, como transferências de *tokens* ou execução de lógicas predefinidas (WU; ZOU; SONG, 2023).

A estrutura de um contrato Solidity é organizada de forma lógica e modular, ela começa com a declaração do *pragma*, que especifica a versão mínima e máxima do compilador Solidity necessária para compilar o contrato. Essa declaração é essencial para evitar incompatibilidades com diferentes versões do compilador e assegurar a consistência na execução do contrato. Logo após, é comum incluir o identificador de licença, uma

prática recomendada que facilita a conformidade com licenças de código aberto e promove a transparência do projeto (BASHIR, 2023; WU; ZOU; SONG, 2023).

A definição do contrato é realizada com a palavra-chave *contract*, que encapsula a lógica e os dados associados. Dentro do contrato, encontramos as variáveis de estado, que são fundamentais para armazenar informações persistentes no *blockchain*. Essas variáveis representam o estado atual do contrato e podem incluir dados como saldos, endereços ou qualquer outra informação relevante para a aplicação. As funções desempenham um papel central na lógica do contrato, permitindo a interação entre usuários e o sistema. Elas podem ser configuradas com diferentes níveis de visibilidade, como pública, privada, interna ou externa, e podem ser marcadas como *payable* para receber transferências da criptomoeda da rede. Além disso, os modificadores de função são utilizados para aplicar restrições ou pré-condições às funções, garantindo, por exemplo, que apenas o proprietário do contrato possa executar determinadas operações (BASHIR, 2023; WU; ZOU; SONG, 2023).

Outro elemento importante são os eventos, que permitem a emissão de *logs* no *blockchain*. Esses *logs* são úteis para notificar usuários ou sistemas externos sobre a ocorrência de eventos específicos, como atualizações de saldo ou transferências. Complementando a estrutura, Solidity oferece suporte a tipos definidos pelo usuário, como *struct* e *enum*, que ajudam a organizar dados complexos de maneira estruturada e intuitiva. Os mapeamentos (*mappings*) são uma poderosa ferramenta de armazenamento que associa pares de chave-valor, frequentemente usados para vincular endereços a informações como saldos ou permissões. Essa flexibilidade na manipulação de dados faz dos mapeamentos um recurso indispensável em contratos inteligentes (BASHIR, 2023; WU; ZOU; SONG, 2023).

Além de sua estrutura modular, para o desenvolvimento de contratos em Solidity segue-se um ciclo bem definido, como ilustrado na Figura 4, sendo ele é fundamental para garantir a integridade e a funcionalidade das soluções implementadas. O ciclo começa com a escrita do código do contrato inteligente em Solidity, em seguida, o contrato é compilado, traduzindo a lógica do contrato em instruções de baixo nível. Após a compilação, o contrato deve então ser testado, com objetivo verificar seu comportamento, em condições controladas, e identificar possíveis falhas assegurando que ele atenda aos requisitos funcionais e de segurança. Somente após os testes o contrato deve ser implantado na rede de destino, de forma a torná-lo acessível para interações reais por usuários e aplicações. Por fim, uma vez implantado, o contrato entra em execução, permitindo que invoquem suas funções e interajam com ele por meio de transações. Este ciclo de desenvolvimento não é apenas uma sequência linear, mas um processo iterativo. Sempre que melhorias ou correções forem necessárias, o ciclo se reinicia, criando-se novas versões do contrato, assegurando que a aplicação se mantenha atualizada e funcional (WU; ZOU; SONG, 2023).

Figura 4 – Ciclo representado o processo de desenvolvimento na linguagem Solidity.

Fonte: Adaptado de (WU; ZOU; SONG, 2023).

2.3 Tecnologia Relacionadas a Criação da Rede *Blockchain*

A construção de uma rede *blockchain* requer ferramentas consolidadas e flexíveis, capazes de atender às demandas de segurança, escalabilidade e integração. Esta seção aborda inicialmente o Hyperledger Besu, uma plataforma focada em redes permissionadas e compatível com o EVM. Em seguida, são explorados os conceitos referentes às APIs de chamada de procedimento remoto em redes *blockchain*, que facilitam a interação e integração entre aplicações e redes *blockchain*. Por fim, discorre-se sobre Docker e Docker Compose, ferramentas indispensáveis para a criação e gestão de ambientes conteinerizados, que proporcionam maior eficiência no desenvolvimento e manutenção da infraestrutura sugerida.

2.3.1 Hyperledger Besu

O Hyperledger é um projeto iniciado em 2015 pela Linux Foundation com o objetivo de desenvolver *frameworks* e plataformas de *blockchain* voltados para o uso empresarial. Diferente das *blockchains* públicas, como Bitcoin ou Ethereum, que são frequentemente associadas a criptomoedas, o Hyperledger busca oferecer soluções que se adequem às necessidades específicas de indústrias e empresas. A iniciativa se baseia na colaboração aberta e no desenvolvimento de *software* de código aberto, promovendo o avanço tecnológico sem duplicação de esforços (LAURENCE, 2019; BASHIR, 2023).

A fundação conta com o suporte de grandes empresas e líderes do setor, como

IBM, Intel, J.P. Morgan e Wells Fargo, bem como de startups inovadoras. Com um modelo de governança robusto, liderado por comitês técnicos, o Hyperledger garante a interoperabilidade e a solidez de suas tecnologias. O projeto utiliza um ciclo de vida de desenvolvimento similar ao da própria Linux Foundation: propostas de novos projetos são submetidas, passam por uma fase de incubação e, quando estáveis, são movidas para um estado ativo (LAURENCE, 2019).

Entre os principais *frameworks* desenvolvidos sob o guarda-chuva do Hyperledger estão o Fabric, liderado pela IBM; o Sawtooth, liderado pela Intel; e o Iroha, desenvolvido pela Soramitsu. Cada um desses *frameworks* atende a casos de uso específicos e fornece ferramentas para a construção de *blockchains* privadas e soluções empresariais (BASHIR, 2023).

O Hyperledger Besu, ou apenas Besu, é um dos principais projetos dentro da fundação Hyperledger. Desenvolvido originalmente pela ConsenSys, o Besu é uma implementação de código aberto da EVM voltada para redes permissionadas e empresariais. Ele foi projetado para oferecer uma alternativa flexível e segura à Ethereum pública, possibilitando a criação de redes privadas de *blockchain* adaptadas às necessidades específicas das empresas (BASHIR, 2023).

Uma das principais características do Besu é sua compatibilidade total com a Ethereum. Isso significa que contratos inteligentes desenvolvidos em Solidity, bem como ferramentas baseadas no ecossistema Ethereum, podem ser utilizados sem alterações. Com essa compatibilidade, soluções inicialmente projetadas para o Ethereum, podem desfrutar de uma migração facilitada para um ambiente mais controlado e privado (BESU, 2024a; WU; ZOU; SONG, 2023).

Outra característica que destaca o Hyperledger Besu é seu suporte a diferentes mecanismos de consenso, principalmente os relacionados aos consensos *Byzantine Fault Tolerance* (BFT), entre eles o IBFT e o QBFT (CAPOCASALE; GOTTA; PERBOLI, 2023). Essa flexibilidade permite que empresas escolham o modelo mais adequado às suas necessidades, considerando fatores como segurança, eficiência e requisitos de governança (BESU, 2024a).

Outro ponto forte do Besu é sua modularidade e extensibilidade, alinhadas aos princípios do Hyperledger. Ele pode ser configurado para operar em diferentes cenários, desde redes permissionadas empresariais até redes públicas compatíveis com Ethereum, oferecendo controle sobre segurança e desempenho (WU; ZOU; SONG, 2023). Além disso, em termos de privacidade, o Besu oferece recursos avançados, como suporte a transações privadas, utilizando ferramentas como Tessera (BESU, 2024b).

2.3.2 APIs de Chamada de Procedimento Remoto em Redes *Blockchain*

Uma API de chamadas de procedimento remoto (RPC), do inglês, *Remote Procedure Call*, é uma interface que permite a execução de operações em um sistema remoto como se fossem chamadas locais. Esse mecanismo possibilita que um cliente envie requisições para um servidor remoto e receba respostas sem precisar lidar diretamente com os detalhes da comunicação subjacente (GOUGH et al., 2022). Em *blockchains*, essas APIs são um meio fundamental de interação entre aplicativos descentralizados, carteiras, e outras aplicações externas com os nós da rede. As chamadas típicas incluem operações para consultar informações do *blockchain*, enviar transações e interagir com contratos inteligentes (SCHEID et al., 2019).

Cada nó de *blockchain* geralmente expõe uma API RPC que serve como uma interface de comunicação. Por exemplo, no Ethereum, a API RPC fornece métodos para verificar o saldo de uma conta, consultar o estado de um contrato inteligente ou enviar uma transação, de maneira que as chamadas têm formato *JavaScript Object Notation* (JSON) e tudo é processado via uma API JSON/RPC. Essa interface padronizada permite que as aplicações se conectem a qualquer nó da rede sem a necessidade de um software proprietário, promovendo a interoperabilidade (SCHEID et al., 2019; LEE et al., 2019).

A importância das APIs RPC na *blockchain* reside em sua capacidade de facilitar a comunicação entre sistemas. Para as aplicações descentralizadas, as APIs RPC são o elo entre o *frontend* (interface do usuário) e a *blockchain* subjacente. Ao invés de operar diretamente no protocolo da rede, as aplicações enviam comandos utilizando RPC, abstraindo métodos computacionais implementados pela rede, possibilitando uma simplificação no desenvolvimento e uniformização das requisições (WU; ZOU; SONG, 2023). Além disso, essas APIs podem atuar no monitoramento e análise de redes, permitindo que ferramentas de exploração de blocos e *dashboards* obtenham informações em tempo real sobre transações, contratos e eventos da *blockchain* (LEE et al., 2019).

Por fim, essas interfaces também têm implicações de segurança e desempenho. A comunicação entre o cliente e o nó RPC precisa ser protegida contra ataques, como interceptações ou modificações de dados (LI et al., 2021).

2.3.3 Docker e Docker-compose

No desenvolvimento de *software* moderno, a busca pela padronização e pela portabilidade tornou-se primordial para enfrentar a diversidade de ambientes onde as aplicações são executadas. Cada sistema pode apresentar configurações distintas, o que aumenta a complexidade de garantir que o *software* funcione de forma uniforme. Nesse contexto, a conteinerização surgiu como uma solução inovadora, permitindo que aplicações sejam empacotadas junto com todas as suas dependências em contêineres isolados. Esse modelo reduz

problemas de compatibilidade e facilita a transição entre diferentes ambientes, promovendo uma experiência consistente para desenvolvedores e usuários (KANE; MATTHIAS, 2023).

Entre as ferramentas que transformaram a forma como aplicações são gerenciadas, o Docker se destacou desde sua introdução em 2013. Essa tecnologia revolucionou a conteinerização ao oferecer uma abordagem simples e eficiente para criar, distribuir e executar aplicações. Os contêineres criados com Docker são leves e portáteis, assegurando que o *software* encapsulado funcione de maneira uniforme, seja em ambientes locais, seja em grandes infraestruturas de nuvem. Esse nível de consistência não apenas melhora a confiabilidade do desenvolvimento, mas também minimiza os custos e os esforços associados à resolução de conflitos de configuração (NICKOLOFF; KUENZLI, 2019).

Enquanto o Docker é ideal para a gestão de contêineres individuais, o Docker Compose foi desenvolvido para lidar com aplicações mais complexas que dependem de múltiplos serviços interconectados. Com o Compose, é possível definir em um único arquivo de configuração, do tipo *Yet Another Markup Language* (YAML), com todos os serviços necessários, incluindo detalhes sobre redes e volumes compartilhados. Essa ferramenta permite que sistemas compostos por diversos contêineres sejam iniciados e gerenciados com um único comando, otimizando tanto o desenvolvimento quanto a implantação de arquiteturas distribuídas (DOCKER, 2024).

2.3.4 Hardhat

O Hardhat é um *framework* amplamente utilizado no desenvolvimento de contratos inteligentes e aplicações descentralizadas (DApps) na plataforma Ethereum. Lançado em 2018 pela Nomic Labs, o Hardhat se destaca por ser um ambiente completo para desenvolver, testar, compilar, implantar e depurar contratos inteligentes, oferecendo recursos que simplificam significativamente o ciclo de desenvolvimento. Construído sobre Node.js, ele suporta linguagens como Solidity e Vyper, ambas compatíveis com ambiente Ethereum (WU; ZOU; SONG, 2023).

Um dos principais componentes do Hardhat é o Hardhat *Runner*, projetado para realizar operações de implantação e gerenciar tarefas específicas por meio de uma *Command Line Interface*(CLI). O Hardhat adota uma arquitetura baseada em tarefas e *plugins*, permitindo que desenvolvedores personalizem seu ambiente de acordo com suas necessidades. Entre os *plugins* mais populares está o hardhat-ethers, que integra a biblioteca ethers.js ao ambiente de execução do Hardhat, possibilitando interações simplificadas com a *blockchain* Ethereum (WU; ZOU; SONG, 2023).

Com uma comunidade ativa e um desenvolvimento contínuo, o Hardhat se consolidou como uma ferramenta importante para o ecossistema Ethereum, especialmente devido à sua capacidade de otimizar o fluxo de trabalho e proporcionar um ambiente seguro para o

desenvolvimento e validação de contratos inteligentes (WU; ZOU; SONG, 2023).

2.4 Tecnologias de Suporte ao Desenvolvimento do Sistema Proposto

O desenvolvimento do sistema proposto, para se conectar com a rede criada, exigiu a utilização de tecnologias complementares que foram organizadas nesta seção para refletir a arquitetura do sistema. Primeiramente, apresenta-se o Fastify, *framework* aplicado no desenvolvimento do *backend* do sistema. Na sequência, React e Vite são explorados, uma vez que são ferramentas úteis para a construção de interfaces dinâmicas e performáticas. Por fim, discute-se brevemente a respeito do PostgreSQL, um banco de dados relacional amplamente reconhecido por sua eficiência e flexibilidade. Esta organização reflete o fluxo natural do desenvolvimento do sistema, conectando o *backend* ao *frontend* e, finalmente, ao banco de dados.

2.4.1 Fastify

O Fastify é um *framework web* baseado em Node.js, projetado para o desenvolvimento de servidores *Hypertext Transfer Protocol* (HTTP) de forma eficiente, escalável e segura. Lançado oficialmente em 2018, o *framework* se destaca por seu foco no desempenho, sendo uma das opções mais rápidas no ecossistema Node.js, e por oferecer uma experiência de desenvolvimento otimizada, com um design modular baseado em *plugins* (SPIGOLON; SINIK; COLLINA, 2023).

Sua arquitetura inclui componentes principais, como a instância principal de aplicação, que gerencia solicitações e respostas no formato HTTP, e objetos específicos, como o *Request* e o *Reply*, que facilitam a interação com dados de entrada e saída. Além disso, componentes utilitários, como *hooks* e *decorators*, permitem a extensão das funcionalidades padrão, promovendo a reutilização de código e a simplicidade no gerenciamento do ciclo de vida das aplicações. Essa flexibilidade torna o Fastify uma solução apropriada para aplicações modernas e escaláveis (SPIGOLON; SINIK; COLLINA, 2023).

2.4.2 React e Vite

React é uma biblioteca JavaScript criada por Jordan Walke, engenheiro de software do Facebook, que foi inicialmente utilizada no feed de notícias do Facebook em 2011 e mais tarde integrada ao Instagram em 2012. Tornada *open source* em 2013, a biblioteca rapidamente conquistou espaço na comunidade de desenvolvimento *web*, sendo descrita como uma solução que se concentra na camada de visualização. Grandes empresas como Uber, Netflix e Airbnb adotaram React por sua eficiência na criação de interfaces dinâmicas

e responsivas, o que solidificou sua posição no mercado como uma ferramenta apreciada para o desenvolvimento de aplicações modernas (BANKS; PORCELLO, 2020).

Um dos diferenciais de React é a simplicidade de sua arquitetura baseada em componentes, que são reutilizáveis e promovem a modularidade no desenvolvimento. Esses componentes permitem que desenvolvedores criem interfaces complexas dividindo a aplicação em partes menores e mais gerenciáveis. Além disso, a introdução de recursos como React Fiber em 2017, que otimizou o algoritmo de renderização, e o advento de *Hooks* em 2019, que trouxe uma maneira elegante de lidar com estado e efeitos colaterais, reforçou a relevância da biblioteca. A abordagem declarativa de React também facilita a manipulação da interface do usuário, permitindo que os desenvolvedores descrevam "o que"a interface deve ser em vez de "como"ela deve ser construída (BANKS; PORCELLO, 2020).

Ainda no contexto de desenvolvimento *web*, o Vite complementa o React como uma ferramenta de empacotamento de módulos rápida e eficiente. Desenvolvido por Evan You, ela foi projetada para resolver limitações de softwares tradicionais, como *Webpack*, ao oferecer um servidor de desenvolvimento baseado em *ESBuild*, que reduz drasticamente o tempo de inicialização e construção das aplicações (VITE, 2024).

Por fim, juntas essas tecnologias oferecem uma experiência de desenvolvimento *frontend* otimizada e modernizada, com React simplificando a lógica da interface e Vite acelerando a configuração e o fluxo de trabalho.

2.4.3 PostgreSQL

O PostgreSQL é um sistema de gerenciamento de banco de dados relacional amplamente reconhecido por sua escalabilidade e conformidade com o padrão da linguagem *Structured Query Language* (SQL). Sua popularidade decorre não apenas de sua natureza de código aberto, mas também de sua capacidade de lidar com dados complexos e grandes volumes de informações. Além disso, sua integração nativa com SQL, um padrão consolidado e amplamente utilizado na manipulação de dados, facilita o desenvolvimento e manutenção de aplicações. Por meio do PostgreSQL, é possível gerenciar dados de maneira eficiente, facilitando a integração e o desenvolvimento de sistemas, ao passo que conta com uma comunidade ativa e uma documentação consolidada para suporte contínuo (FERRARI; PIROZZI, 2023).

2.5 Trabalhos Relacionados

O avanço das tecnologias de *blockchain* e IoT tem gerado um impacto significativo em aplicações relacionadas à IoV, especialmente em áreas como compartilhamento seguro

de dados, rastreabilidade e integração de sistemas descentralizados. Desse modo, esta seção aborda estudos que exploram a aplicação dessas tecnologias no contexto de transporte inteligente e veículos conectados, propondo soluções que envolvem segurança, privacidade e eficiência no gerenciamento de dados veiculares.

No trabalho de FERDOUS et al. (2020), foi proposta uma arquitetura baseada em *blockchain* para criar um registro imutável de dados gerados por carros inteligentes, denominado "autobiografia do carro". O objetivo é abordar desafios relacionados à confiança nos dados, armazenamento seguro e compartilhamento de informações de forma amigável à privacidade. A solução garante a imutabilidade e a integridade dos dados por meio da *blockchain* e contratos inteligentes, permitindo que informações sejam compartilhadas entre diferentes partes interessadas, como proprietários de veículos, fabricantes e agências governamentais. Em sua abordagem, os autores consideraram utilizar, em propostas futuras, a ferramenta Hyperledger Fabric para criar uma *blockchain* privada, mas não destacaram um sistema que auxilie o usuário se comunicar diretamente com a rede, embora os autores abordem sobre a importância de manter seguras as informações obtidas através de dados advindos dos carros.

Por sua vez, WANG; ZHANG (2020) desenvolveram um esquema de compartilhamento seguro de dados e serviços personalizados para sistemas de transporte inteligente utilizando *blockchain* de consórcio, utilizando uma autoridade confiável. A proposta incluiu um algoritmo de re-encriptação *proxy* baseado em atributos, que permitia a busca por palavras-chave em dados criptografados, ao mesmo tempo que controlava o acesso e protegia a privacidade. Além disso, contratos inteligentes foram utilizados para oferecer serviços personalizados, como precificação de seguros e manutenção de veículos, com análises de desempenho, em ambiente simulado, demonstrando vantagens em termos de segurança e eficiência computacional. Com relação à coleta de dados, os autores propuseram a criação de uma unidade embarcada que era responsável por enviar os dados coletados diretamente dos veículos. No entanto, para que a *blockchain* e todo o sistema proposto funcionem adequadamente, não basta apenas a presença do veículo; foi necessária também uma infraestrutura de suporte nas vias, composta por unidades fixas estrategicamente posicionadas para auxiliar na comunicação e transmissão dos dados. Ademais, embora em sua abordagem mostrem como a *blockchain* pode transformar processos tradicionais em cidades e transportes inteligentes, eliminando vulnerabilidades associadas à centralização, os autores não detalharam uma plataforma para criação e gestão da *blockchain*, tampouco demonstraram aplicações em um cenário real.

DWIVEDI et al. (2020) propôs uma arquitetura descentralizada baseada em *blockchain* para compartilhar informações críticas de eventos, como congestionamentos e acidentes, em redes veiculares ad-hoc. A solução incluiu um protocolo de autenticação para garantir a confiabilidade das mensagens e um mecanismo de consenso para validar tran-

sações. O estudo também projetou contratos inteligentes para automatizar a troca de informações entre veículos e unidades à beira da estrada. A análise de segurança destacou que o protocolo é efetivo contra ataques e atende a requisitos como integridade e anonimato, promovendo confiança e transparência no compartilhamento de informações em ambientes de IoV. Para o sistema proposto os autores utilizaram uma plataforma em nuvem que armazena os dados, assumindo que essa plataforma pertencia a um proprietário confiável.

Com base nas pesquisas apresentadas, é evidente que a integração de *blockchain* com sistemas de transporte inteligente, como a IoV, é um campo de estudo em constante expansão, com contribuições significativas relacionadas à segurança, rastreabilidade e privacidade no gerenciamento de dados veiculares. No entanto, observa-se que as soluções existentes deixam lacunas importantes. Por exemplo, embora Ferdous et al. (2020) aborda a criação de uma *blockchain* privada para registro imutável de dados veiculares, a ausência de um sistema completo para intermediar a comunicação direta entre os veículos e a rede pode limitar o alcance prático da solução. Da mesma forma, Wang e Zhang (2020) destacam o uso de uma unidade embarcada para coleta de dados, mas não detalham uma plataforma robusta para a criação e gestão da *blockchain*, nem abordam mecanismos de interação direta do usuário com a rede.

Nesse contexto, o presente trabalho propõe uma solução integrada que aborda as limitações identificadas nos estudos analisados. Enquanto abordagens anteriores exploram o uso de *blockchain* para registro de dados veiculares, elas frequentemente não contemplam uma infraestrutura completa que viabilize a comunicação direta entre os dispositivos e a rede. Assim, a solução apresentada neste estudo se diferencia ao integrar uma *blockchain* privada com um sistema que não apenas incorpora a coleta de dados via *hardware*, mas também assegura o armazenamento seguro das informações. Para isso, é estabelecida uma intermediação eficiente entre dispositivos IoT e a *blockchain*, possibilitando a interação direta do usuário por meio de uma interface gráfica e de uma API construída com tecnologias emergentes. Além disso, a solução proposta é validada através de uma prova de conceito, na qual um cenário real é utilizado para demonstrar sua viabilidade prática, consolidando a segurança e a rastreabilidade dos dados veiculares.

3 ABORDAGEM PROPOSTA

Neste capítulo, será apresentada a metodologia empregada no desenvolvimento de um sistema para registro de dados veiculares na *blockchain*, bem como o processo para o desenvolvimento de uma rede *blockchain* privada utilizando a ferramenta Hyperleder Besu. O objetivo é descrever a arquitetura geral do sistema proposto, abordando a integração entre os componentes responsáveis pela coleta, processamento, visualização e armazenamento seguro dos dados coletados durante determinada rota.

A implementação foi idealizada para certificar o funcionamento de todo sistema, desde a coleta dos dados até a consolidação deles em registros imutáveis na rede privada, buscando garantir a rastreabilidade e acessibilidade confiável das informações, atendendo às necessidades de uma plataforma que visa proporcionar aplicabilidade em cenários reais de gerenciamento das informações em veículos conectados.

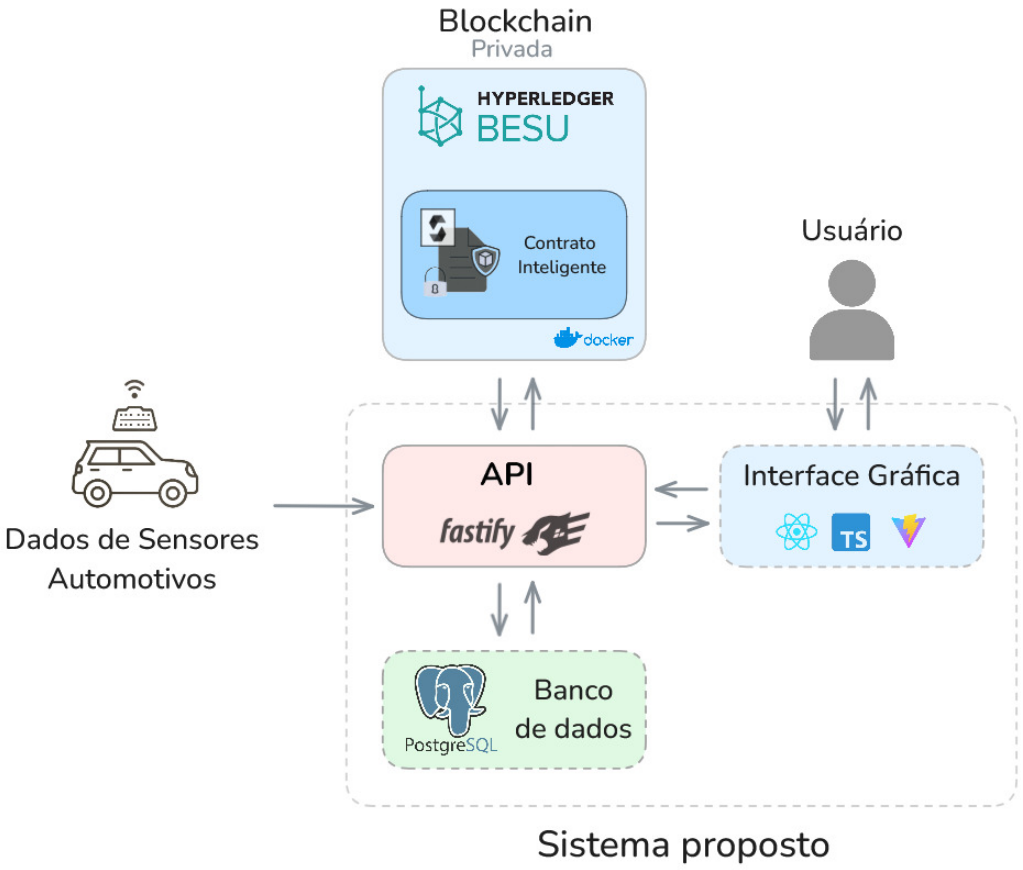
Para sua viabilidade, o sistema proposto integra tecnologias de modo a compor uma arquitetura modular. O fluxo de dados começa com a coleta nos sensores automotivos, que enviam as informações à uma API. Após o processamento inicial, os dados são armazenados temporariamente no banco de dados e, mediante requisição, enviados para a *blockchain*. A interface gráfica permite que os usuários acompanhem suas viagens realizadas e efetivem as operações de registro, interagindo diretamente com a *blockchain*.

Conforme ilustrado na Figura 5, o ponto principiante de todo funcionamento da solução se evidencia na coleta de sensores instalados no veículo. Esses sensores extraem informações relevantes relacionadas à viagem realizada pelo motorista, como localização, a quantidade de rotações por minuto (RPM), velocidade do veículo, carga do motor e outras métricas.

Uma vez que os dados são coletados, adentra-se no funcionamento da arquitetura planeada. Através de uma conexão direta entre o veículo e o sistema, uma API atua como intermediária entre os dados coletados e os outros componentes do sistema. Sua principal função é receber os dados enviados pelos sensores automotivos, processá-los e integrá-los ao sistema. Além disso, a API gerencia a comunicação bidirecional entre a interface gráfica, o banco de dados e a rede *blockchain*, firmando-se como porção medular de toda metodologia aplicada.

A rede *blockchain* privada, configurada utilizando o Hyperledger Besu, é responsável pelo armazenamento imutável dos dados críticos de cada viagem. Através do uso de contratos inteligentes, os dados são validados e registrados na *blockchain*, garantindo sua integridade, rastreabilidade e segurança. Utiliza-se o consenso QBFT para assegurar

Figura 5 – Visão geral da abordagem proposta para solução de registro de dados veiculares em blockchain.



Fonte: Elaborada pelo autor, 2025.

confiabilidade no processamento das transações dentro da rede proposta.

O banco de dados é utilizado para o armazenamento temporário e organizado dos dados veiculares. Essa camada permite que os dados sejam indexados, consultados e visualizados de forma eficiente antes de serem efetivados à *blockchain* para registro permanente. Por fim, a interface gráfica do sistema proposto proporciona uma experiência de usuário interativa, permitindo consultas aos registros processados, a visualização de resumos de viagens consolidados na *blockchain* e a interação com o sistema de maneira prática.

As demais seções deste capítulo estão dedicadas a descrever atentamente o desenvolvimento de cada um dos componentes que foram brevemente apresentados nesta visão geral da solução proposta.

3.1 Configuração da Rede *Blockchain* com Hyperledger Besu

A rede *blockchain* utilizada foi configurada com base no Hyperledger Besu, uma plataforma de código aberto projetada para a implementação de redes *blockchain* compatíveis com o ecossistema Ethereum. A escolha do Hyperledger Besu foi motivada principalmente por sua adequação a redes privadas e seu suporte completo à infraestrutura do Ethereum, incluindo a EVM e a linguagem de programação Solidity. Essa compatibilidade permite a compilação e execução dos contratos inteligentes, ampliando a flexibilidade e a praticidade de desenvolvimento da rede.

Devido ao suporte para redes privadas, o Hyperledger Besu oferece também algoritmos de consenso adequados para aplicações privadas, como o QBFT, escolhido para a rede configurada neste trabalho. Este algoritmo permite uma maior segurança e tolerância a falhas em redes privadas, proporcionando uma melhor adequação ao cenário de aplicação proposto para a rede.

Além disso, a plataforma oferece configurações que permitem a personalização da rede para atender a aplicações específicas, como a possibilidade de configurar uma rede sem a necessidade de taxas de gás. Esse grau de customização pode ser alcançado por meio da definição de parâmetros ajustados antes da criação da rede.

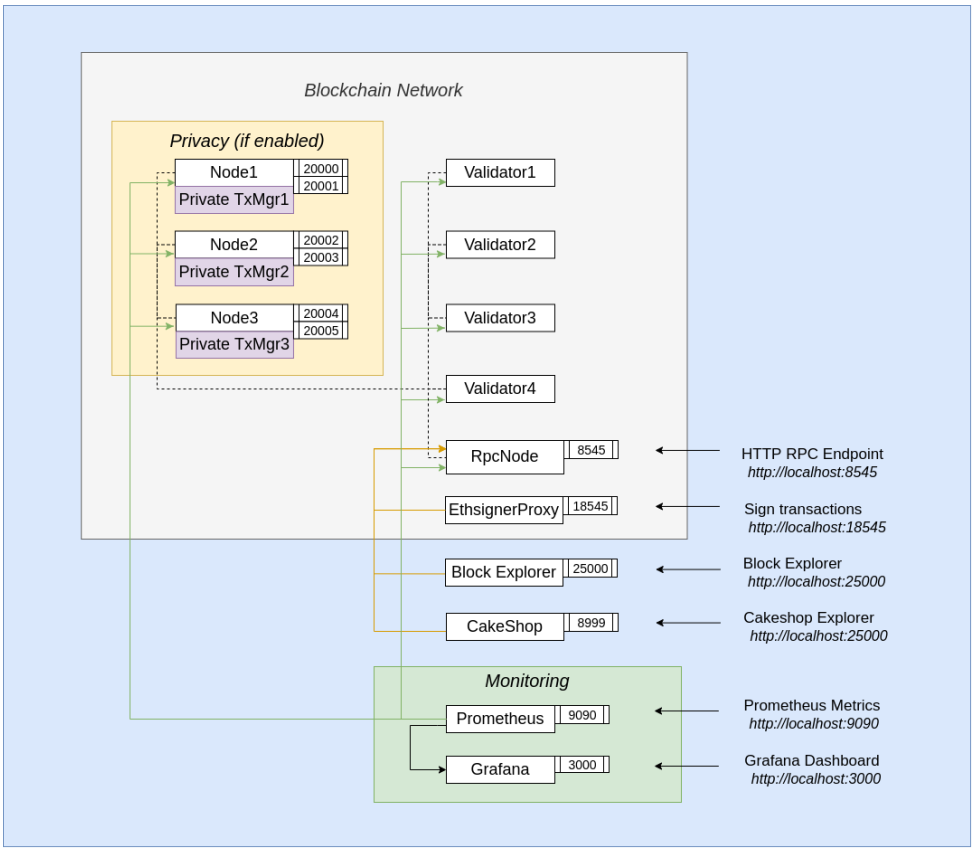
Inicialmente, a configuração da rede é baseada no *Quorum Developer Quickstart*, um recurso pré-configurado que simplifica a criação de redes *blockchain* para desenvolvimento e testes na plataforma Hyperledger Besu. Previamente a configuração da rede local, faz-se necessária a instalação do Java, do Node.js e do Docker no computador que irá hospedá-la.

Uma vez que os pré-requisitos são atendidos, ao se executar o *quickstart*, optou-se pela criação de uma rede privada, devido à sua maior adequação para ambientes controlados. Com como resultado temos a criação dos arquivos e pastas necessários para se executar uma rede utilizando imagens Docker do Hyperledger Besu, gerenciadas por meio de um arquivo Docker Compose também disponibilizado. Essa estrutura inicial contém configurações e arquivos necessários para a inicialização de uma rede *blockchain* privada composta por diversos nós, cada um desempenhando funções específicas, de forma a totalizar o funcionamento da rede.

Como ilustrado na Figura 6, uma grande quantidade de componentes é criada, sendo cada um deles um container Docker. Além disso, percebe-se que há um pré configuração atrelada a cada um desses container, junto com respectivos mecanismos de comunicação via rede local. Com intuito de esclarecer esses componentes, destacam-se as principais características de cada um configurado pelo *quickstart*, inicialmente:

1. Validadores (Validator1, Validator2, Validator3, Validator4): Nós que são responsáveis por validar transações e criar blocos na *blockchain*. No *quickstart* são criados quatro

Figura 6 – Arquitetura de *Containers* criados para Rede *Blockchain* com o Hyperledger Besu *quickstart*.



Fonte: Hyperledger besu, 2024.

validadores por padrão, número mínimo de validadores para a rede operar com segurança, de acordo com o funcionamento do consenso QBFT. Cada validador tem seu respectivo container docker e é configurado com chaves criptográficas, participando ativamente do processo de consenso.

2. Nó RPC (RPCNode): Este nó atua como ponto de entrada para a interação externa com a rede. Ele expõe APIs que permitem às aplicações clientes enviarem transações, consultar dados da *blockchain* e interagir com contratos inteligentes.

3. Nós de Membros Replicadores (Node1, Node2, Node3): São nós que mantêm uma cópia completa do ledger da *blockchain*. Garantem redundância e maior segurança para a rede. Por padrão, no *quickstart* são configurados três *containers* docker, cada um representando um membro pertencente a rede. Nota-se que nessa configuração há suporte também à privacidade transacional, para os membros, por meio do uso da ferramenta Tessera.

   - Tessera: Tecnologia que essencialmente implementa funcionalidades de privacidade transacional, permitindo transações privadas entre participantes da rede

(BESU, 2024b).

4. Ferramentas de Monitoramento: A pré-configuração inclui em sua rede *containers* docker dedicados a monitoramento, com plataformas como Prometheus, Grafana e o Quorum Explorer, que fornecem métricas detalhadas e interfaces gráficas para o *status* da rede.

Com esse conjunto de ferramentas, mais de treze *containers* docker são levantados, isso pois as ferramentas de monitoramento requerem outros *containers* auxiliares para garantir seu funcionamento. Dessa forma, faz-se necessário que a máquina responsável por executar a rede seja capaz de lidar com uma demanda computacional que suporte toda infraestrutura.

3.2 Idealização do contrato inteligente

Posterior a criação da rede *blockchain*, propõe-se a criação de um contrato inteligente cuja função principal será armazenar informações sobre viagens veiculares de forma rastreável. Esse contrato será implementado na rede *blockchain* privada configurada, aproveitando suas características de imutabilidade e confiabilidade dos dados.

São duas as funcionalidades principais propostas para o contrato. A primeira sendo uma função que permita armazenar os dados de uma viagem. Ela aceitará como entrada informações como a localização de início e término, a distância percorrida, métricas do desempenho do veículo (como velocidade média e aceleração média), além dos *timestamps* correspondentes aos horários de início, término e registro da viagem. O registro será associado a um identificador único da viagem e ao endereço do usuário responsável pelo envio dos dados, garantindo que cada viagem seja vinculada de forma rastreável ao seu proprietário.

Para o caráter único das viagens armazenadas, será implementado um contador de identificadores que será automaticamente incrementado a cada nova viagem registrada. Esse contador facilitará a associação de cada conjunto de dados a um identificador único (ID), permitindo consultas e operações organizadas.

A segunda funcionalidade será uma função de consulta que permitirá recuperar os dados de uma viagem específica a partir do identificador único. Essa função devolverá todas as informações armazenadas, sendo útil para acessar os dados de forma rastreável e auditável. Para essa função é fundamental incluir verificações que garantam consultas apenas a viagens previamente registradas, evitando erros ou acessos inválidos.

Outro aspecto importante será a utilização de eventos dentro do contrato. Sempre que uma nova viagem for armazenada, um evento será emitido, possibilitando que sistemas

externos sejam notificados dessa interação em tempo real. Essa funcionalidade será primordial para integrar o contrato inteligente à API do sistema, permitindo o monitoramento contínuo da utilização do contrato.

Por fim, uma abordagem modular e extensível deve ser adotada para o desenvolvimento do contrato, permitindo ajustes futuros para incorporar novas funcionalidades ou otimizações. Com essa proposta, a rede terá um componente responsável por assegurar que os dados veiculares coletados sejam armazenados e gerenciados de forma segura, contribuindo para análises posteriores e aplicações relacionadas.

3.3 Configuração do Hardhat

Para interagir com a rede *blockchain* configurada e realizar a implantação (*deploy*) dos contratos inteligentes, utiliza-se o *framework* Hardhat, reconhecido por sua flexibilidade e eficiência no desenvolvimento de contratos em Solidity. A utilização do Hardhat no presente projeto é fundamental, permitindo não apenas uma conexão com a rede, mas proporcionando todo um ambiente para testes e aperfeiçoamento do contrato.

A configuração proposta foi idealizada para atender aos requisitos do sistema, incluindo suporte a redes locais, compatibilidade com a rede privada baseada no Besu e a isenção de taxas de gás, viabilizando eficiência nos testes e implementações.

Com Hardhat é possível realizar a compilação de contratos escritos em Solidity, sendo necessário apenas informar qual a versão utilizada pelo contrato. Além disso, visando uma possível melhora no desempenho e redução de custos computacionais na execução dos contratos, pode-se ativar o otimizador do compilador, sendo necessário apenas informar o número de execuções idealizadas para a otimização. Essa configuração pode ser útil para contratos que exigem interações repetitivas, como o armazenamento de dados veiculares e consultas frequentes, evitando um *overhead* computacional desnecessário.

Aprofundando o intuito da utilização no *framework* neste projeto, como ilustrado na Figura 7, após o desenvolvimento do contrato com a linguagem Solidity o Hardhat foi utilizado para compilar o contrato de acordo com a versão escolhida e para execução de scripts de teste. Após obter sucesso nos testes unitários, utilizou-se o mesmo *framework* para realizar o *deploy* do contrato na rede criada, obtendo assim informações úteis para a execução das funcionalidades da API relacionadas ao contrato inteligente, essas informações são compiladas automaticamente em um arquivo denominado *contract.json*.

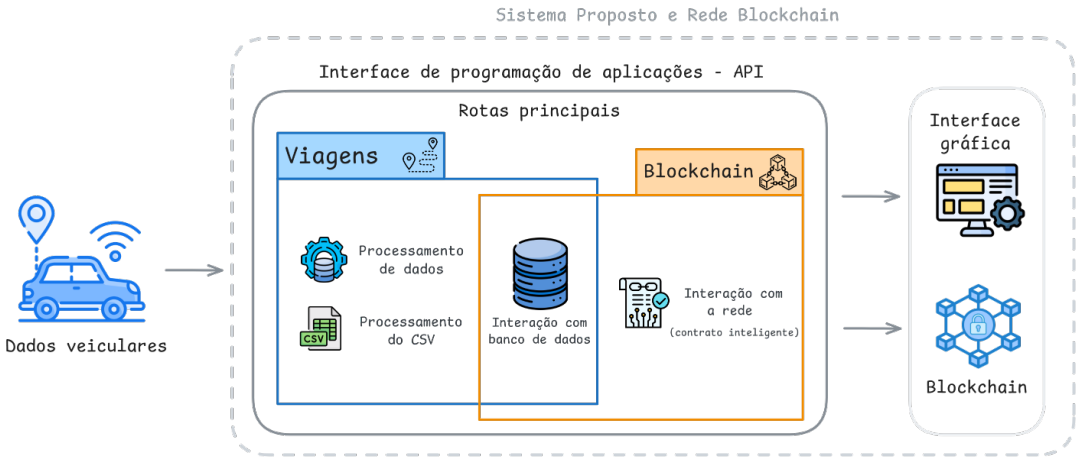
Figura 7 – Fluxo detalhado da aplicação do Hardhat na Solução.

Fonte: Elaborada pelo autor, 2025.

3.4 Elaboração e funcionamento da API

Uma vez que todos os requisitos relacionados ao *blockchain* foram atendidos, o próximo elemento fundamental a ser desenvolvido é a API, que será responsável por conectar os dados provenientes das viagens veiculares à rede criada. O projeto da API foi organizado em uma estrutura que facilita a modularização e manutenção do código. Dessa forma, ela foi estruturada de maneira a ter dois segmentos principais, cada um contendo as respectivas funções de utilidade destinadas a auxiliar na finalidade primária. Como ilustrado na Figura 8, esses dois segmentos mencionados pertencem às principais rotas da interface, sendo elas denominadas *blockchain* e viagens que, respectivamente, gerenciam as rotas relacionadas às funcionalidades da *blockchain* e ao armazenamento de dados de viagens.

Figura 8 – Visão Geral da proposta para da API a ser elaborada.



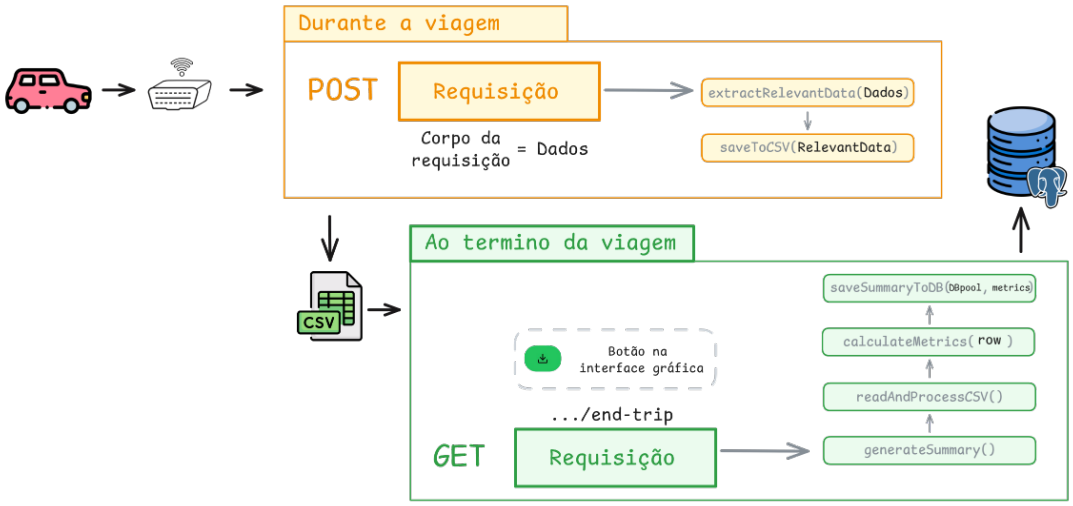
Fonte: Elaborada pelo autor, 2025.

O arquivo principal da API é tido como ponto de entrada do sistema, onde o servidor Fastify é configurado e iniciado. Para isso, nele estão especificadas as variáveis de ambiente para definir parâmetros, como a porta do servidor, e integrar *middlewares* como o *fastifyCors*, evitando problemas relacionados a políticas de *Cross-origin resource*

sharing (CORS). Ainda no arquivo do servidor, são registradas as rotas dos arquivos responsáveis pelos segmentos mencionados, sendo eles o de viagens e o de *blockchain*. Além disso, com objetivo de tratar qualquer tipo de conteúdo recebido em requisições, um *parser* é adicionado. Antes de iniciar o servidor, o código testa a conexão com o banco de dados, garantindo que a API só seja iniciada se a conexão for bem sucedida.

Como ilustrado na Figura 9, são definidas rotas para gerenciar os dados das viagens. Dentre as suas funcionalidades, por exemplo, há uma rota *POST* que recebe dados temporários de um veículo e os salva em um arquivo *comma-separated value* (CSV) usando a função *saveToCSV* do módulo de manipulação do CSV. Outra funcionalidade, executada por meio de uma rota *GET*, permite o usuário interagir com a interface gráfica do sistema, de modo que, ao acionar um botão uma requisição é gerada para que uma viagem seja processada, ou seja, finalizada. Com isso, os dados do CSV são usados e processados, gerando métricas, como distância percorrida, velocidade média, e outros parâmetros calculados pelas funções auxiliares referentes ao processamento dos dados.

Figura 9 – Detalhamento do segmento da API destinado a viagem realizadas pelos veículos.



Fonte: Elaborada pelo autor, 2025.

Com objetivo de municiar o funcionamento do segmento relacionado a viagens, destaca-se as principais funções do módulo referente ao processamento dos dados e, posteriormente, do módulo destinado a manipulação do arquivo CSV.

- *extractRelevantData()*: função responsável por extrair informações relevantes do corpo de uma requisição com base em um mapeamento de chaves prefinidas, como velocidade, RPM e carga do motor.
- *calculateMetrics()*: função que processa os dados armazenados em CSV para calcular métricas como distância percorrida, velocidades média e total, RPM médio, e outros parâmetros.

- *saveToCSV()*: função que recebe os dados processados e os escreve em um arquivo temporário.
- *readAndProcessCSV()*: lê os dados do arquivo temporário e os prepara para serem processados pelas funções de processamento. Sendo utilizada principalmente ao final de uma viagem.

Para o seu funcionamento correto a função *generateSummary()* engloba a chamada das funções *readAndProcessCSV()* e *calculateMetrics()*, respectivamente.

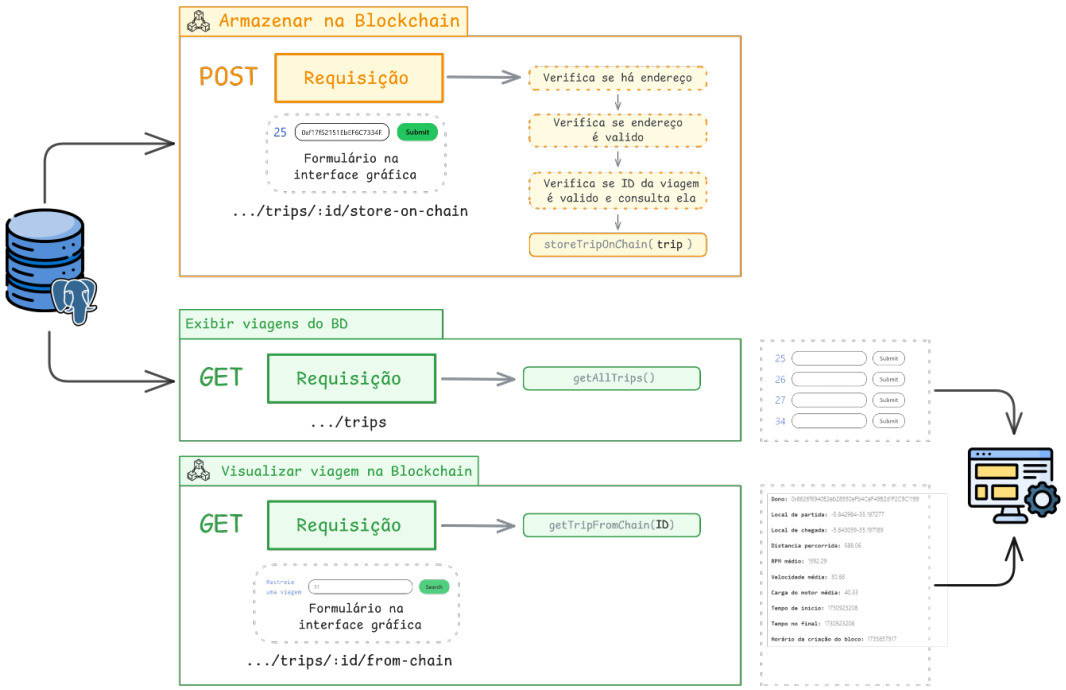
Por fim, há um módulo de utilidades atreladas ao banco de dados, que concentra as operações e as ações de gerência relacionadas ao PostgreSQL. Apesar da sua ligação com as duas rotas, sua aplicação está mais atada à rota Viagens, com apenas uma funcionalidade particular na rota *Blockchain*. Dando seguimento a descrição deste módulo, ele compreende funções que realizam consultas ao banco e para isso o módulo compõe duas funções, sendo elas:

- *saveSummaryToDatabase()*: salva no banco de dados o resumo, gerado ao final de uma viagem. Ela é aplicada principalmente após a obtenção das métricas por meio do CSV.
- *getAllTrips()*: responsável por retornar todas as viagens armazenadas em formato JSON. Utilizada para exibição dos dados na interface gráfica da aplicação, como ilustrado na Figura 10.

Seguindo adiante, as rotas da API que lidam diretamente com a interação entre o sistema e a *blockchain* (Figura 10) são responsáveis por armazenar viagens na *blockchain* e buscar informações de viagens já registradas. Para isso, em consequência do funcionamento do contrato criado, sua primeira funcionalidade, implementada por meio de uma rota *POST*, permite enviar os dados de uma viagem armazenada no banco de dados para a *blockchain*. Desse modo, em sua requisição, é exigido que o cliente forneça o ID da viagem (no banco) e o endereço Ethereum do usuário. A rota realiza diversas validações, como verificar se o ID é válido e se o endereço Ethereum segue o formato correto. Em seguida, por meio de uma conexão com o banco de dados, ela busca os dados da viagem e caso ela seja encontrada será armazenada na *blockchain*.

Já sua segunda funcionalidade, tem o desígnio de buscar informações de uma viagem que já foi armazenada na *blockchain*. Ela recebe o ID da viagem (na *blockchain*) como parâmetro e valida se o ID é válido. Após a validação, há uma consulta ao contrato inteligente para recuperar os dados da viagem associada ao ID fornecido. Caso os dados sejam encontrados, a função retorna um objeto contendo informações como o proprietário da viagem e as informações dela. Esses dados são enviados como resposta ao cliente no

Figura 10 – Detalhamento do segmento da API destinado a interagir com a *Blockchain*.



Fonte: Elaborada pelo autor, 2025.

formato JSON e, após uma formatação, exibidos na interface gráfica. Caso a viagem não seja encontrada ou ocorra um erro durante o processo, a rota deve informar, especificando se o problema foi a ausência do ID na *blockchain* ou uma falha técnica.

Para interação com a rede, o módulo de suporte criado para a rota *Blockchain* utiliza o *framework ethers.js* para se conectar a um contrato inteligente previamente configurado. Ele define a função *storeTripOnChain*, que envia uma transação contendo os dados da viagem para o contrato. A função aguarda a confirmação da transação e verifica se o evento esperado, chamado *TripStored*, está presente nos *logs* da transação. Esse evento é emitido pelo contrato inteligente ao armazenar os dados com sucesso. Caso o evento seja encontrado, a função extrai informações relevantes, como o *hash* da transação, o ID da viagem gerado na *blockchain*, o endereço do proprietário da viagem, e outros dados relacionados. Caso o evento não seja encontrado ou ocorra algum erro durante a transação, a função lança uma exceção para que a API possa tratar o erro.

A outra função ainda nesse módulo, chamada *getTripFromChain*, utiliza o método *getTrip* do contrato inteligente para buscar os dados de uma viagem já registrada. A função verifica se os dados retornados contêm os campos esperados, como o endereço do proprietário e as métricas da viagem, e, em seguida, os formata para serem enviados como resposta pela API. Em caso de falha na consulta, como um ID inexistente ou erros na conexão com a *blockchain*, a função gera uma mensagem de erro detalhada para facilitar a identificação do problema.

3.5 Banco de dados

O banco de dados utilizado na aplicação é o PostgreSQL, um sistema gerenciador de banco de dados objeto relacional. Sua escolha foi motivada por sua simplicidade, prestígio e aceitação diante do abundante número de aplicações que o utilizam. Embora a *blockchain* seja o principal componente de armazenamento para garantir integridade e rastreabilidade dos dados, o uso de um banco de dados relacional como o PostgreSQL complementa a solução, oferecendo maior flexibilidade para consultas rápidas e análises estruturadas dos dados.

Para solução desenvolvida, o banco armazena informações detalhadas das viagens em uma tabela, incluindo campos que integram e condizem com o funcionamento tanto da API desenvolvida quanto do contrato inteligente como localizações inicial e final, distância total percorrida, velocidade média e outras informações.

A integração entre o banco de dados e a *blockchain* permite combinar duas características dessas soluções: o banco de dados relacional oferece acesso rápido e eficiente para dados operacionais enquanto a *blockchain* permite a rastreabilidade e a imutabilidade de dados críticos. Esse modelo híbrido visa refletir as necessidades de aplicações reais, onde a escalabilidade e usabilidade são tão importantes quanto a segurança e confiabilidade.

Como apresentado nas etapas anteriores deste capítulo, o banco de dados está ativamente presente no desenvolvimento da solução proposta, pertencendo desde as fases iniciais do processamento dos dados até que eles sejam efetivamente registrados no *blockchain*, destacando-se portanto o papel estratégico do PostgreSQL na aplicação.

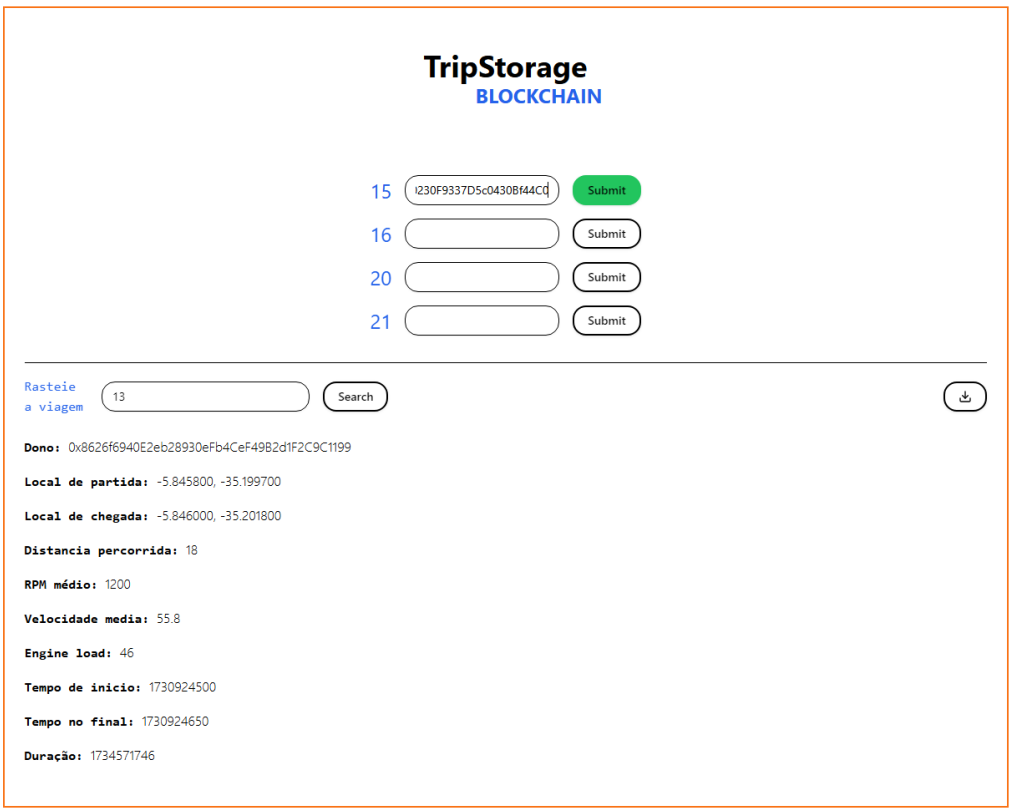
3.6 Interface de interação com usuário

A interface de interação do usuário com a solução de armazenamento de dados de viagens na *blockchain* foi desenvolvida utilizando React, um *framework* JavaScript amplamente reconhecido por sua capacidade de construir interfaces dinâmicas e responsivas. A escolha do React permite gerenciar estados e eventos de forma eficiente, favorecendo a criação de componentes reutilizáveis e a manutenção do código.

Como ilustrado na Figura 11, a interface oferece uma experiência de usuário voltada para rastreamento e registro de viagens, sendo composta por dois blocos principais: o formulário de registro e a área de busca e exibição de dados de viagens.

A primeira parte da interface apresenta um formulário de submissão, por meio do qual é possível associar o endereço público de um usuário, pertencente à rede, a uma viagem, presente no banco de dados, e armazenar as informações diretamente na *blockchain*. Esta funcionalidade é implementada por uma função, que realiza uma requisição HTTP *POST* para um *endpoint* da API local. Nota-se que para que haja o registro da viagem na

Figura 11 – Visualização da interface de usuário proposta para a Solução.



Fonte: Elaborada pelo autor, 2025.

blockchain criada, este campo de entrada faz-se necessário para a submissão do formulário. A execução bem-sucedida é sinalizada por um *feedback* visual, fornecido pelo componente de *toast* da aplicação, implementado através do *hook* personalizado *useToast()*.

Além disso, o *frontend* oferece uma funcionalidade de rastreamento de viagens, permitindo que o usuário pesquise os dados armazenados na *blockchain* a partir do identificador da viagem. A busca é realizada de modo a realizar uma requisição HTTP *GET* para recuperar informações detalhadas, como local de partida, local de chegada, distância percorrida e outros parâmetros relevantes, exibidos dinamicamente na interface.

Uma característica adicional da aplicação é a possibilidade de realizar a requisição para gerar o resumo da viagem caso exista um arquivo CSV contendo informações consolidadas. Essa funcionalidade também se apoia em uma requisição HTTP *GET* para um *endpoint* da API.

Além das funcionalidades mencionadas, a interface implementa um processo contínuo de sincronização de dados com o *backend*, utilizando o *hook useEffect()* para carregar automaticamente a lista de viagens disponíveis, através de uma requisição à API, implementada por uma função mencionada em seções anteriores. Esse recurso garante que as informações exibidas ao usuário estejam sempre atualizadas, melhorando a interatividade

do sistema.

4 EXPERIMENTOS E RESULTADOS

Neste capítulo, são descritos e apresentados os resultados obtidos a partir da implementação da rede privada e do sistema apresentado na abordagem proposta. Três tópicos principais são abordados. Inicialmente é apresentado os primeiros testes com o contrato, fora da rede *blockchain*. Conduzidos via o *framework* Hardhat, por meio de testes unitários foi possível atestar o funcionamento correto do contrato.

Em seguida, as primeiras interações com a rede criada são abordadas, demonstrando seu funcionamento através da interface do explorador de blocos e com o *deploy* do contrato desenvolvido, proporcionado via Hardhat.

Por fim, com propósito de validar toda solução abordada anteriormente, uma prova de conceito foi conduzida através de uma viagem em um cenário real, com a rede e todo o sistema desenvolvido em comunicação via internet.

4.1 Validação do contrato Inteligente

Anteriormente ao *deploy* e utilização do contrato na rede *blockchain* criada, seguindo os passos para desenvolvimento de contratos inteligentes com Solidity (Figura 4), um *script* em Javascript foi elaborado para testar as funcionalidades do contrato desenvolvido de maneira unitária. Para executar o script e realizar os testes necessários utiliza-se do *framework* Hardhat. Os testes estão divididos em duas partes principais:

1. *Store Trip*: Verifica se uma viagem é armazenada corretamente, incluindo a emissão do evento *TripStored*, contendo os valores esperados, ou seja, os dados de uma viagem armazenada correspondem às informações fornecidas como entrada.

2. *Get Trip*: Confirma que uma viagem armazenada pode ser recuperada pelo ID e que os dados retornados são precisos. Também testa o comportamento do contrato ao tentar acessar uma viagem inexistente, garantindo que ele retorne um erro adequado.

A estrutura do código foi elaborada de maneira que o contrato é implementado antes de cada teste, assegurando consistência e isolamento entre os casos de avaliação. O uso de *timestamps* e validação de eventos reforça a integridade dos testes, cobrindo cenários reais de uso e erro. Para seu funcionamento adequado, dados fictícios foram utilizados como entrada. A Figura 12 ilustra o resultado dos testes realizados com o contrato, bem como o tempo de execução deles, exceto o último, em que o tempo de duração não excedeu 25ms.

Figura 12 – Resultados dos testes realizados com o contrato inteligente proposto.

```
TripStorage
  Store Trip
    ✓ Should store a trip and emit a TripStored event (862ms)
  Get Trip
    ✓ Should retrieve the correct trip data by ID (52ms)
    ✓ Should revert if the trip does not exist


3 passing (935ms)
```

Fonte: Elaborada pelo autor, 2025.

4.2 Implementação da Rede

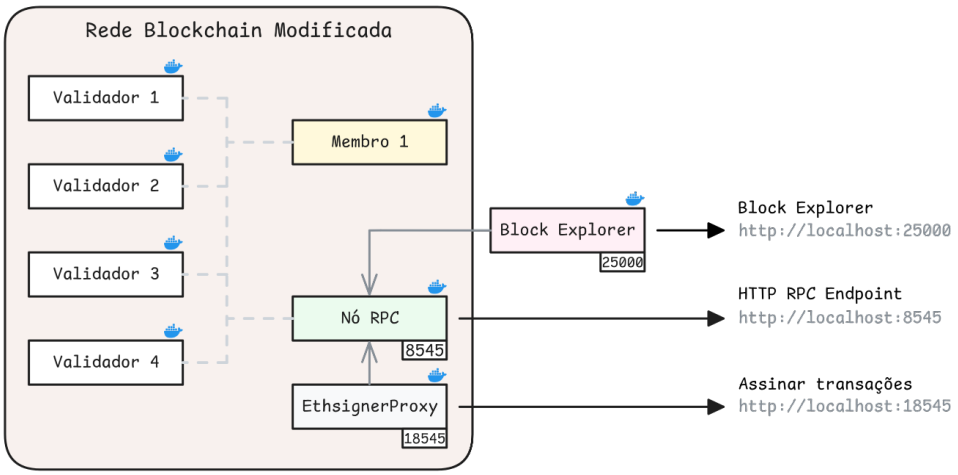
A implementação da rede *blockchain* privada utilizando o Hyperledger Besu foi realizada seguindo o arquivo *docker-compose.yml* dispobilizado pelo quickstart. Embora o *quickstart* ofereça uma configuração inicial abrangente, muitas dessas funcionalidades não são essenciais para os objetivos deste trabalho. Assim, foi realizada uma customização significativa no arquivo.

4.2.1 Modificações na rede *quickstart*

Com o objetivo de otimizar os recursos necessários para a infraestrutura da rede, as modificações realizadas buscaram reduzir a complexidade e focar principalmente nas funções essenciais da *blockchain*.

A customização resultante resultou em uma infraestrutura mais enxuta e objetiva, mantendo apenas os elementos fundamentais, conforme ilustrado na Figura 13.

Figura 13 – Arquitetura de *Containers* criados para Rede *Blockchain* com o Hyperledger Besu, após modificações.



Fonte: Elaborada pelo autor, 2025.

As primeiras alterações estão relacionadas a remoção de *containers* não necessários, sendo essa mudança iniciada pelos *containers* dedicados ao monitoramento da rede. Foram removidas ferramentas como Prometheus, Grafana e todos os *containers* ligados a essas aplicações. Contrário aos anteriores, o explorador de blocos Quorum Explorer, foi mantido como a única ferramenta de monitoramento, pois oferece uma interface amigável para acompanhar blocos registrados, transações confirmadas e a atividade dos validadores, se demonstrando apropriado para perceber o funcionamento e as interações com a rede.

Seguindo as alterações mencionadas, o número de membros replicadores foi reduzido para um único nó. O objetivo desta escolha é simplificar a infraestrutura sem comprometer a funcionalidade da rede, uma vez que para a proposta preliminar da rede privada, não se faz necessário tantos nós replicadores. Esta alteração também visa reduzir a carga computacional de manter a rede em funcionamento.

Junto aos nós replicadores, toda infraestrutura do Tessera e o suporte a transações e grupos privados foi removido. Essa decisão foi entendida como válida pois não há necessidade de implementar modelos privados uma vez que para solução desenvolvida todos os participantes são conhecidos.

Além da remoção de ferramentas e aplicações julgadas não essenciais, para o funcionamento da rede da maneira desejada, alguns parâmetros foram removidos, a saber a remoção de dependências não mais necessárias, como as associadas a grupos privados, mencionados anteriormente. Por outro lado, outros parâmetros foram acrescentados, sendo o principal relacionado a tornar a rede sem custo de gás. Para isso, nas configurações iniciais de todos os nós responsáveis por executar foi ajustada a definição do preço para o gás, de forma a tornar a mínima possível, ou seja, zero.

4.2.2 Bloco gênesis da *blockchain* privada

Além dos container responsáveis pela execução e manutenção da rede, para especificar o funcionamento dela, faz-se necessário uma configuração anterior ao seu funcionamento. Essas especificações dependem de um arquivo JSON que define o bloco gênesis da rede, devendo ele ser configurado antes da sua inicialização e devendo ser uniformemente replicado em todos os nós participantes.

A configuração do arquivo gênesis é uma etapa fundamental na criação de qualquer rede *blockchain*, incluindo as redes baseadas no Hyperledger Besu. Nesse processo são estabelecidos os parâmetros iniciais e as regras que governam o funcionamento da rede. Isso pois o arquivo contém uma série de parâmetros que influenciam desde a definição das especificidades para o mecanismo de consenso escolhido até a personalização do comportamento da *blockchain* para casos de uso específicos. Além disso, é importante ressaltar que qualquer discrepância nesse arquivo pode causar falhas de sincronização ou

mesmo isolar nós da rede.

O arquivo contém diversos parâmetros que permitem configurar os detalhes da rede. Enquanto alguns são essenciais para o funcionamento adequado, outros podem ser dispensados sem impactar a operação.

Além disso, no que se refere ao bloco gênesis da rede construída a escolha dos parâmetros foi feita de maneira adequada, conforme informações e orientações apresentadas nos tutoriais disponíveis no site oficial do Hyperledger Besu (BESU, 2024a) e de maneira que a rede melhor se adequasse a proposta desejada. Sendo assim os parâmetros que constituem o bloco gênesis para a rede são os seguintes:

1. Configurações Gerais (*config*): Parâmetro principal que agrupa as configurações essenciais da rede.
    - *chainId*: Define o identificador único da cadeia *blockchain*, indispensável para diferenciar redes e evitar conflitos. Esse parâmetro também é utilizado por ferramentas como o Hardhat, permitindo que a rede seja reconhecida em testes e interações.
    - Blocos de ativação (*homesteadBlock, daoForkBlock*, etc.): Especificam os blocos em que melhorias ou mudanças de protocolo, introduzidas ao longo do tempo no Ethereum, são ativadas. Essas atualizações incluem desde aprimoramentos básicos de segurança até novos recursos funcionais.
    - *zeroBaseFee*: Configuração que elimina a taxa base, permitindo que a rede funcione sem custos de gás.
2. Parametros do Consenso QBFT
    - *qbft.epochlength*: Define o número de blocos após os quais ocorre a rotação das chaves dos validadores, garantindo maior segurança e descentralização. Para a rede criada, manteve-se o valor definido pelo *quickstart*.
    - *qbft.blockperiodseconds*: Intervalo de tempo entre a criação de blocos, ajustado para atender às necessidades específicas da aplicação.
    - *qbft.requesttimeoutseconds*: Tempo limite para que os nós validadores respondam durante o processo de consenso, prevenindo atrasos ou travamentos.
3. Parâmetros de Rede e Bloco Inicial
    - "*timestamp*": Marca o momento de criação do bloco gênesis e serve como referência temporal para os blocos subsequentes.

- "*gasLimit*": Define o limite máximo de gás por bloco. Para a rede privada criada, esse valor foi configurado como extremamente alto, eliminando restrições e facilitando o desenvolvimento.

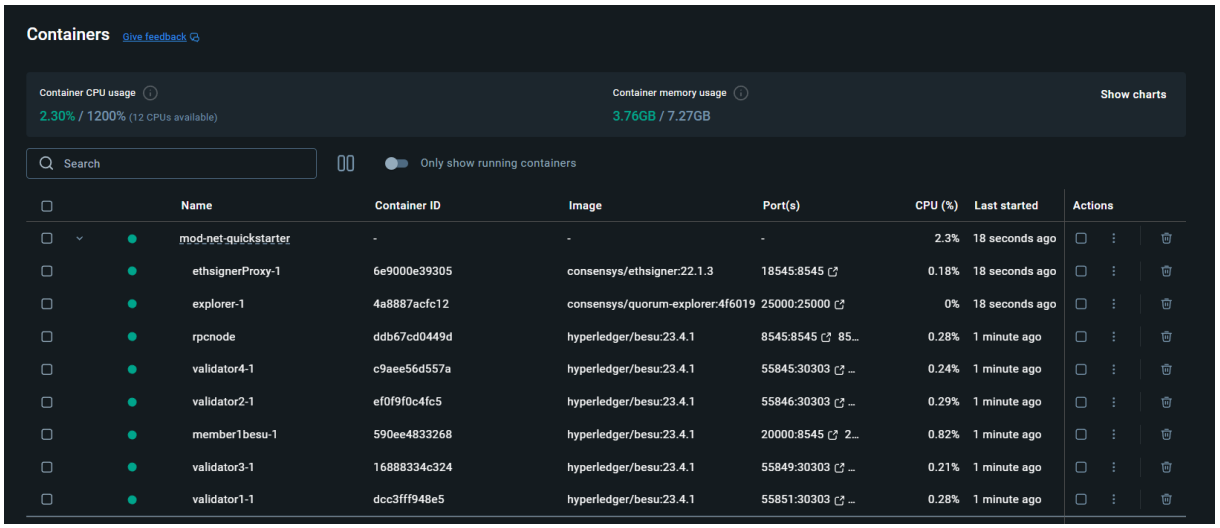
4. Alocações Iniciais de Recursos (*alloc*): Essa seção permite atribuir saldos iniciais a endereços específicos. Isso é útil para desenvolvimento e testes, permitindo a criação de contas predefinidas com saldos significativos, facilitando a simulação de cenários reais. Para a rede criada, foram estabelecidos três endereços de teste, todos com saldo elevado, a fim de evitar qualquer limitação relacionada a recursos monetários durante os testes.

4.2.3 Validando o funcionamento da *blockchain* privada

Dessa forma, após a configuração de um *script* Shell para automatizar os comandos necessários à execução do Docker Compose, a rede foi iniciada.

Conforme o esperado, a estrutura proposta foi validada por meio da execução de múltiplos *containers* em Docker (totalizando oito) utilizando uma configuração mínima e otimizada dos containers necessários para o funcionamento da rede. Conforme ilustrado na Figura 14, os containers em execução incluem os destacados na Figura 13.

Figura 14 – Visualização, pelo Docker Desktop, dos containers da rede em execução.



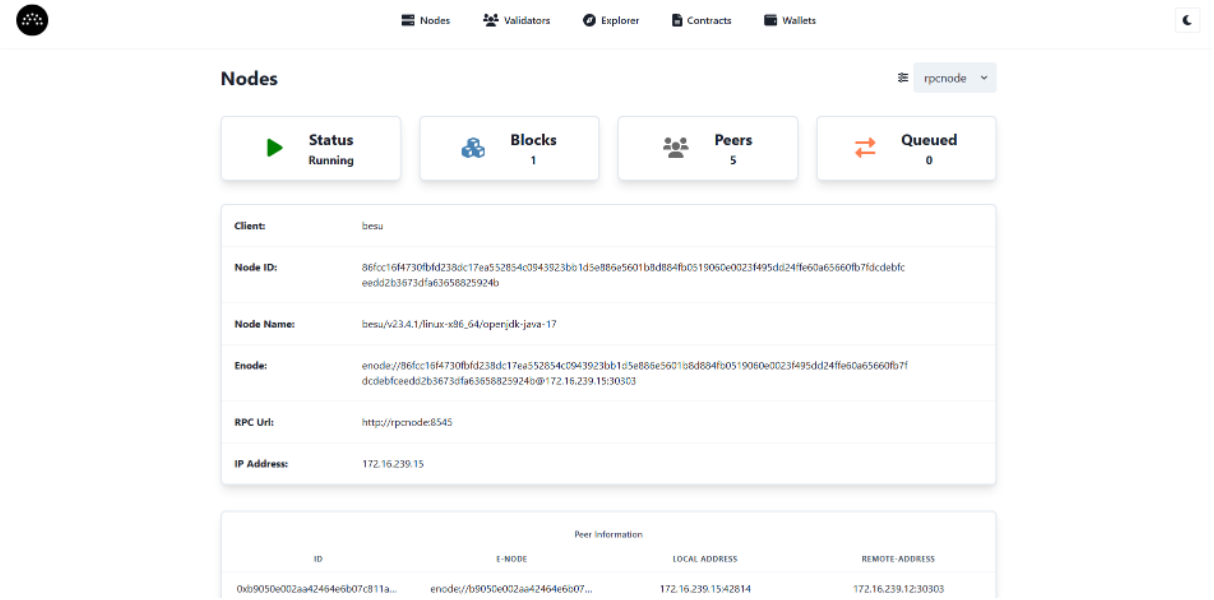
Fonte: Elaborada pelo autor, 2025.

Observou-se, conforme o monitoramento no Docker Desktop, que a carga do processador permaneceu baixa. Ademais, embora a eficiência da abordagem, o uso de memória por parte dos *containers* e seus processos ficou em torno de 3.7 GB.

Dando continuidade aos primeiros testes de funcionamento da rede, para visualizar seu funcionamento, utilizou-se o Quorum Explorer. A Figura 15 apresenta a interface

do explorador de blocos, destacando as informações a respeito da rede. A primeira a se destacar é a quantidade de nós (*peers*) conectados, sendo ao todo cinco *peers* ativos, comprovando a conectividade e o consenso entre nós. É possível visualizar também o *status* da fila de transações, nesse estágio inicial da rede, nenhuma transação está pendente para ser, indicando que a rede está processando as operações necessárias para seu funcionamento. Além dessas informações a interface inicial do explorador, disponibiliza outras métricas dos *peers*, em destaque alguns dados sobre a sincronização do nó RPC principal.

Figura 15 – Interface inicial do explorador de blocos.



Fonte: Elaborada pelo autor, 2025.

Para concluir a validação da rede e garantir que todos os nós estão corretamente conectados, foi implantado o contrato inteligente criado. De modo que com a implantação sendo bem sucedida, há mais uma confirmação do funcionamento adequado da rede.

O processo foi conduzido por meio da ferramenta Hardhat, utilizando a configuração apresentada na Figura 16. Nessa configuração, a rede privada, denominada *quickstart*, foi configurada para interagir com o nó RPC local do Hyperledger Besu, acessado através do *endpoint http://127.0.0.1:8545*, que corresponde a API RPC da rede. Para garantir que o sistema opere sem custos de transação, é importante definir o preço do gás (parametro *gasPrice*) como zero e desativar a base inicial de taxas (*initialBaseFeePerGas*).

Ainda no que diz respeito a configuração da ferramenta Hardhat, foram especificadas contas pré-configuradas com chaves privadas, que servem como carteiras apenas para teste, simulando diferentes usuários e proprietários de contratos durante testes realizados na rede. Constata-se que essas contas foram as mesmas configuradas no *quickstart*.

Com o Hardhat devidamente configurado, para realizar a implantação do contrato,

Figura 16 – Configuração para utilização do *framework* Hardhat.

```
import { HardhatUserConfig } from "hardhat/config";
import "@nomicfoundation/hardhat-toolbox";

module.exports = {
  networks: {
    quickstart: {
      url: "http://127.0.0.1:8545",
      chainId: 2610,
      gasPrice: 0,
      initialBaseFeePerGas: 0,
      accounts: [
        "0x8f2...be63",
        "0xc87...c0d3",
        "0xae6...162f"
      ]
    }
  },
  defaultNetwork: "hardhat",
  solidity: {
    version: "0.8.19",
    settings: {
      optimizer: {
        enabled: true,
        runs: 200
      }
    }
  },
  paths: {
    sources: "./contracts",
    tests: "./test",
    cache: "./cache",
    artifacts: "./artifacts"
  },
  mocha: {
    timeout: 40000
  }
}
```

Fonte: Elaborada pelo autor, 2025.

um comando apropriado para *deploy* deve ser utilizado. Nesse comando, deve-se constar o caminho para o *script* que realiza o *deploy*, bem como uma identificação da rede de destino.

Sendo assim, a Figura 17 apresenta o comando utilizado para o *deploy*, bem como o resultado que é exibido no terminal de cada nó participante da rede, confirmando o recebimento bem-sucedido da transação.

Adicionalmente, conforme ilustrado na Figura 18, é possível observar o conteúdo gerado no arquivo *contract.json*, uma vez que o *script* de *deploy* do contrato é executado. Conforme explorado anteriormente, o arquivo compilar informações extremamente importantes, uma vez que através delas a API do sistema consegue executar as ações atribuidas as funções do contrato.

Por fim, utilizou-se o o explorador de blocos para verificar a presença do contrato implantado na *blockchain*. Na Figura 19, é possível observar os detalhes do contrato, incluindo o *hash* da transação de implantação, o endereço do contrato e o bloco no qual

Figura 17 – Comando utilizado para o *deploy* do contrato e demonstração do recebimento do contrato por parte da rede através dos *logs* dos blocos da rede.

```
npx hardhat run ./scripts/deploy_tripStorage.js --network quickstart
```

```
2025-01-03 21:12:17 validator4-1      | {"timestamp":"2025-01-04T00:12:17,073","container":"c9aee56d557a","level":"INFO","thread":"BftProcessorE
xecutor-QBFT-0","class":"QbftBesuControllerBuilder","message":"Imported #273 / 1 tx / 0 pending / 933,771 (0.0%) gas / (0xcbb057a723813c122738fc
d8d0e22d9009b72964283017a93680148ab8833ba6)","throwable":""}
2025-01-03 21:12:17 validator1-1      | {"timestamp":"2025-01-04T00:12:17,074","container":"dcc3fff948e5","level":"INFO","thread":"BftProcessorE
xecutor-QBFT-0","class":"QbftBesuControllerBuilder","message":"Produced #273 / 1 tx / 0 pending / 933,771 (0.0%) gas / (0xcbb057a723813c122738fc
d8d0e22d9009b72964283017a93680148ab8833ba6)","throwable":""}
2025-01-03 21:12:17 validator2-1      | {"timestamp":"2025-01-04T00:12:17,082","container":"ef0f9f0c4fc5","level":"INFO","thread":"BftProcessorE
xecutor-QBFT-0","class":"QbftBesuControllerBuilder","message":"Imported #273 / 1 tx / 0 pending / 933,771 (0.0%) gas / (0xcbb057a723813c122738fc
d8d0e22d9009b72964283017a93680148ab8833ba6)","throwable":""}
2025-01-03 21:12:17 validator3-1      | {"timestamp":"2025-01-04T00:12:17,083","container":"16888334c324","level":"INFO","thread":"BftProcessorE
xecutor-QBFT-0","class":"QbftBesuControllerBuilder","message":"Imported #273 / 1 tx / 0 pending / 933,771 (0.0%) gas / (0xcbb057a723813c122738fc
d8d0e22d9009b72964283017a93680148ab8833ba6)","throwable":""}
2025-01-03 21:12:17 rpcnode           | {"timestamp":"2025-01-04T00:12:17,092","container":"ddb67cd0449d","level":"INFO","thread":"EthScheduler-
Workers-0","class":"PersistBlockTask","message":"Imported #273 / 1 tx / 0 om / 933,771 (0.0%) gas / (0xcbb057a723813c122738fcd8d0e22d9009b729642
83017a93680148ab8833ba6) in 0.013s. Peers: 5","throwable":""}
2025-01-03 21:12:17 member1besu-1     | {"timestamp":"2025-01-04T00:12:17,114","container":"590ee4833268","level":"INFO","thread":"EthScheduler-
Workers-0","class":"PersistBlockTask","message":"Imported #273 / 1 tx / 0 om / 933,771 (0.0%) gas / (0xcbb057a723813c122738fcd8d0e22d9009b729642
83017a93680148ab8833ba6) in 0.032s. Peers: 5","throwable":""}
```

Fonte: Elaborada pelo autor, 2025.

Figura 18 – Informações obtidas no arquivo *contract.json* após o contrato ser compilado.


```
{
  "network": "quickstart",
  "contract": {
    "address": "0x988397f1B0FEcD3a1a40CdD5E8221Fa461890517",
    "signerAddress": "0xFE3B557E8Fb62b89F49168721be55cEb828d8d73",
    "abi": [
      "event TripStored(uint256 tripId, address tripOwner, string distance, uint256 startTime, string startLocation, uint256 timestamp)",
      "function getTrip(uint256 _tripId) view returns (address tripOwner, string startLocation, string endLocation, string distance, string avgRPM, string avgSpeed, string avgEngLoad, uint256 startTime, uint256 endTime, uint256 timestamp)",
      "function storeTrip(string _startLocation, string _endLocation, string _distance, string _avgRPM, string _avgSpeed, string _avgEngLoad, uint256 _startTime, uint256 _endTime)",
      "function tripCounter() view returns (uint256)",
      "function trips(uint256) view returns (uint256 tripId, address tripOwner, string startLocation, string endLocation, string distance, string avgRPM, string avgSpeed, string avgEngLoad, uint256 startTime, uint256 endTime, uint256 timestamp)"
    ]
  }
}
```


Fonte: Elaborada pelo autor, 2025.

foi registrado. Esses dados confirmam que o contrato foi devidamente armazenado e está acessível na rede privada.

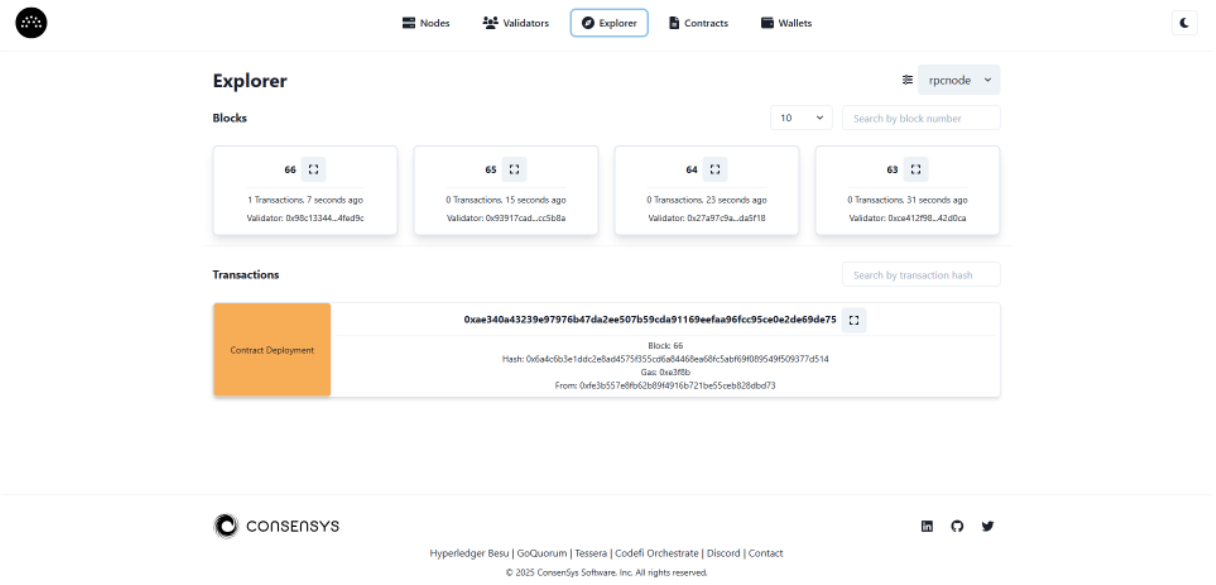
4.3 Prova de Conceito

Para validar a funcionalidade do todo sistema proposto, foi conduzida uma prova de conceito através de uma viagem realizada utilizando um veículo de modelo Fiat Fastback. Esse teste visou demonstrar a coleta, transmissão e armazenamento de dados veiculares na rede *blockchain* privada implementada, atestando sua capacidade de operar em um cenário realista.

4.3.1 Instrumentação

Para esta prova de conceito, utilizou-se o dispositivo Freematics ONE+ como ferramenta para coleta e transmissão de dados veiculares. A escolha do dispositivo foi motivada por três razões: pela sua disponibilidade física, sua natureza open-source e por

Figura 19 – Transação de *deploy* do contrato visualizada no explorador de blocos.



Fonte: Elaborada pelo autor, 2025.

sua viabilidade de utilização de *Wi-fi*. Essas características o tornam adaptável e aceitável para o propósito do experimento.

O Freematics ONE+ é equipado com um microcontrolador baseado em ESP32, que possui dois núcleos e 16 MB de memória *flash*. Este dispositivo é programável em linguagens C/C++ e integra um *System on a Chip* (SoC), permitindo sua operação de forma independente de um *smartphone*. Dessa forma, o dispositivo é capaz de coletar dados diretamente dos sensores veiculares, realizar processamento na borda da rede e transmitir as informações para um servidor através de sua conectividade integrada. Adicionalmente, o dispositivo permite o armazenamento local de dados por meio de cartões microSD de até 32GB.

Supletivo a sua funcionalidade de comunicação via *wi-fi*, destacam-se o suporte a outras tecnologias de comunicação como *Bluetooth* e 4G. Ademais, o *hardware* escolhido dispõe de recursos de geolocalização *Global Navigation Satellite System* (GNSS), útil para obter localização aproximada do carro durante movimento. Para este estudo, foi utilizado o modelo H do Freematics ONE+, que se diferencia dos modelos A e B por suportar tensões de 12V e 24V e por incluir o módulo 4G como padrão. A Figura 20 apresenta a representação física do dispositivo utilizado.

4.3.2 Rota Utilizada

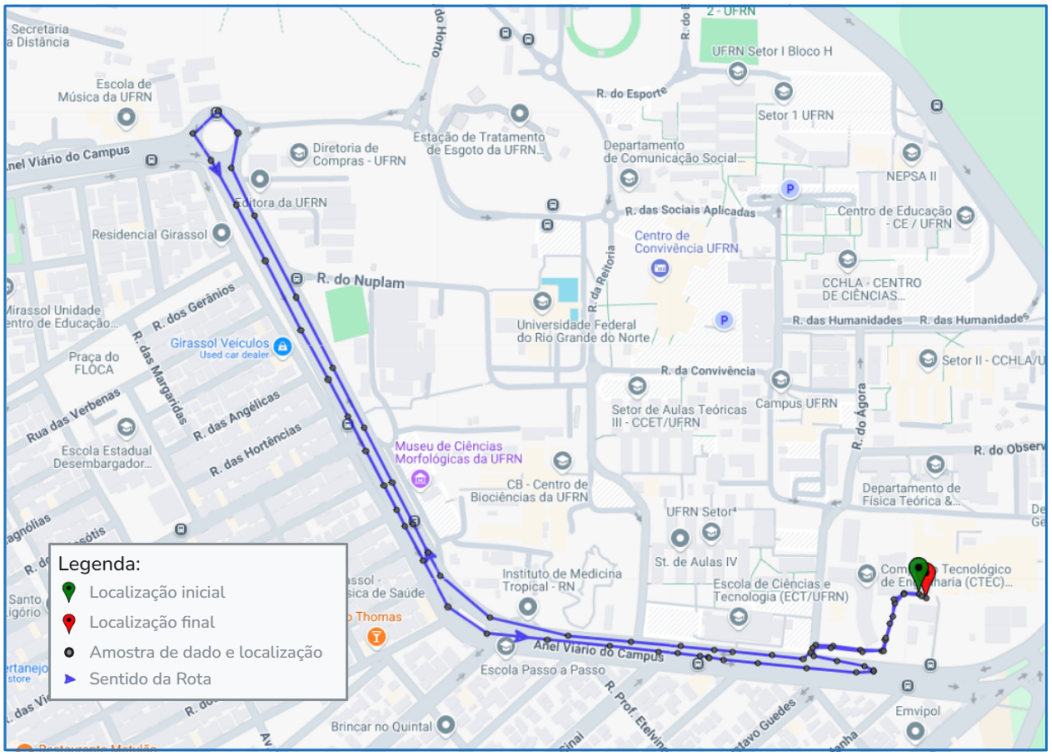
Para a rota utilizada nesta prova de conceito, realizou-se uma viagem dentro do anel viário da Universidade Federal do Rio Grande do Norte. Pela Figura 21, é possível

Figura 20 – Dispositivo Freematics ONE+.

Fonte: Freematics, 2025.

perceber a rota por completo, sendo destacado que ela teve início a finalização no mesmo local, sendo esse o Departamento de Engenharia de Computação e Automação da mesma universidade.

Figura 21 – Mapa da viagem realização.



Fonte: Elaborada pelo autor, 2025.

Complementando a descrição da imagem, os pontos situados no desenho do trajeto referem-se às amostras de dados coletadas do veículo, recebidas pela API e escritas no arquivo CSV. Por fim, pelo desenho traçado é possível identificar o sentido da rota através

das flechas exibidas no circuito ilustrado, indicando respectivamente 25%, 50% e 75% da viagem realizada.

4.3.3 Validação da Solução

A arquitetura do sistema foi configurada para operar de maneira integrada, conectando todos os componentes à mesma rede *Wi-fi*, garantindo a comunicação entre a *blockchain* privada, a API, o banco de dados, a interface gráfica e o dispositivo Freematics ONE+. O contrato inteligente, previamente validado quanto ao seu funcionamento, foi implantado na rede *blockchain*, permitindo o registro das viagens. Com todos os componentes devidamente configurados e comunicando-se entre si, todo sistema apresentou plena funcionalidade para os objetivos do estudo.

Durante o experimento, a API recebeu corretamente os dados enviados pelo dispositivo Freematics ONE+. Após receber o corpo da requisição, a API extraiu as informações relevantes e as armazenou em um arquivo CSV (Figura 22).

Figura 22 – Parte do CSV processado pela API.

```
1  Time Session,Engine Load,RPM,Speed,Latitude,Longitude
2  1730923208,27,1609,12,-5.842984,-35.197277
3  1730923208,26,1441,16,-5.842978,-35.197449
4  1730923208,23,1618,14,-5.843087,-35.197559
5  1730923208,30,1062,16,-5.843324,-35.197617
6  1730923208,32,1021,14,-5.843519,-35.197674
7  1730923208,33,1466,11,-5.843648,-35.197716
8  1730923208,44,1499,25,-5.843636,-35.197960
9  1730923208,23,1491,28,-5.843592,-35.198288
```

Fonte: Elaborada pelo autor, 2025.

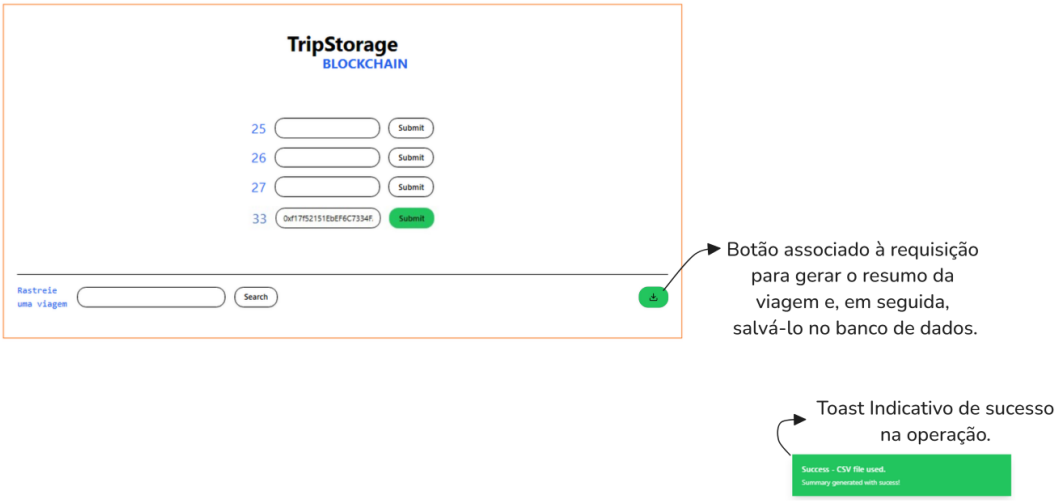
Ao término da viagem, como ilustrado na Figura 23, o botão na interface gráfica para gerar o resumo da viagem foi acionado, e um *toast* de sucesso foi exibido, confirmando o armazenamento bem-sucedido dos dados.

Em seguida, os dados da viagem foram registrados no banco de dados (Figura 24) e apareceram na lista de viagens exibida na interface gráfica (Figura **??**). Prosseguindo com o experimento, o próximo passo foi informar o endereço de um usuário previamente cadastrado na rede *blockchain* para reivindicar e registrar a viagem na *blockchain*. Um endereço de teste foi utilizado para essa operação. Após o envio da transação, outro *toast* foi exibido na interface, indicando o sucesso da operação.

Botão para gerar resumo da viagem e salvá-lo no banco. Indicativo de sucesso na operação.

Para complementar a validação, utilizando o Quorum Explorer, foi possível visualizar que a transação foi corretamente processada pela rede *blockchain*, conforme está destacado na Figura, 26. Pela Figura é possível perceber que são apresentados os detalhes da transação,

Figura 23 – Procedimento de geração do resumo na interface.



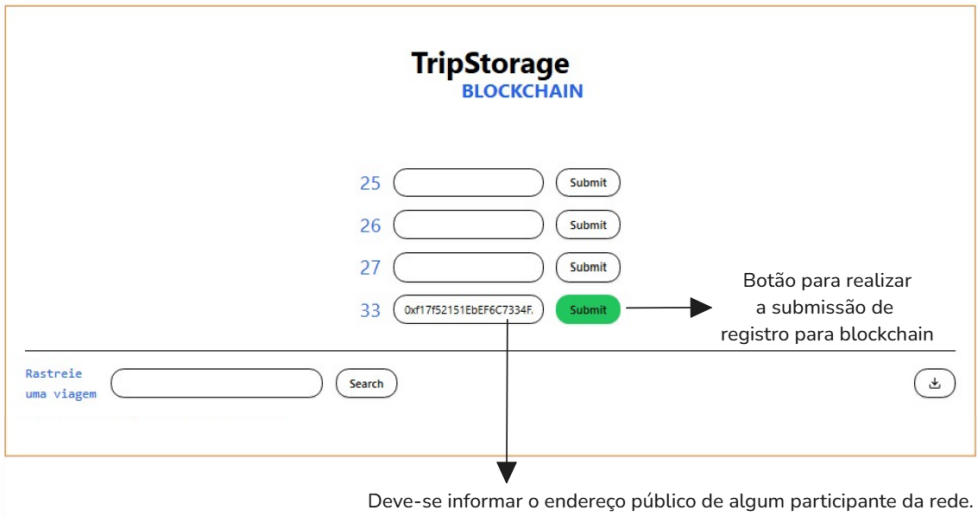
Fonte: Elaborada pelo autor, 2025.

Figura 24 – Viagem registrada no banco de dados.

	id [PK] integer	user_address character varying (255)	start_location character varying (255)	end_location character varying (255)	distance double precision	avg_speed double precision	avg_rpm double precision	avg_engine_load double precision	start_time character varying (50)	end_time character varying (50)
1	25	[null]	-5.840000, -35.198000	-5.845000, -35.206000	15	50.12	1103.5	45	1730923600	1730923700
2	26	[null]	-5.847611, -35.201049	-5.846978, -35.203699	20	60	1155.3	48	1730923800	1730923950
3	27	[null]	-5.843200, -35.198800	-5.842000, -35.201600	12.5	54.7	1190.1	50	1730924000	1730924150
4	33	[null]	-5.842984-35.197277	-5.843039-35.197189	588.06	30.68	1592.29	40.33	1730923208	1730923208

Fonte: Elaborada pelo autor, 2025.

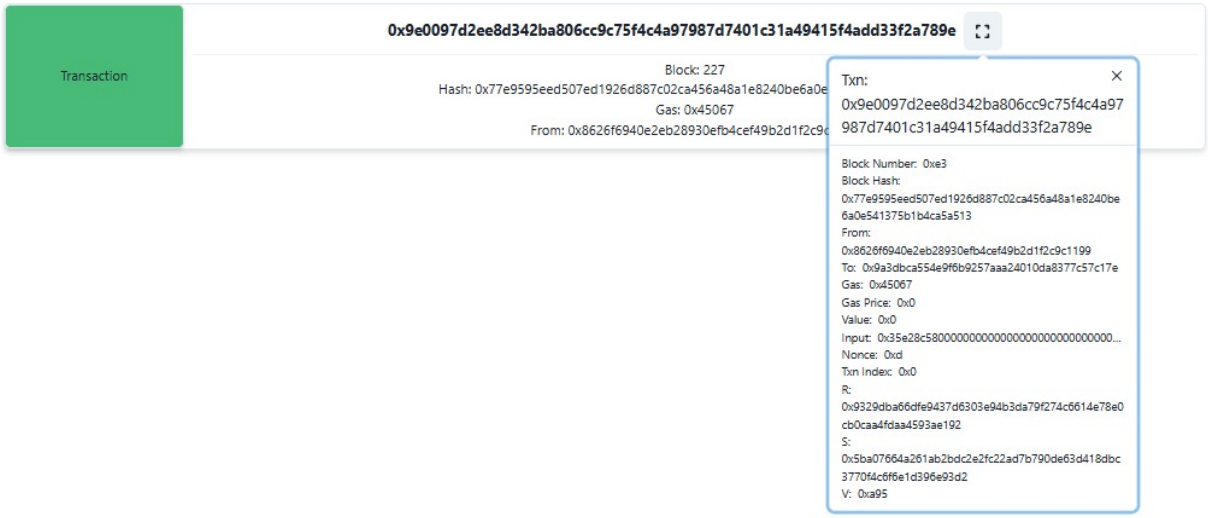
Figura 25 – Procedimento para submissão de viagem do banco para *blockchain*.



Fonte: Elaborada pelo autor, 2025.

assim como o valor do preço do gás consumido, resultando em zero, confirmando a eficiência do registro na rede sem custo ao usuário.

Figura 26 – Transação de registro da viagem na *blockchain* e suas informações.



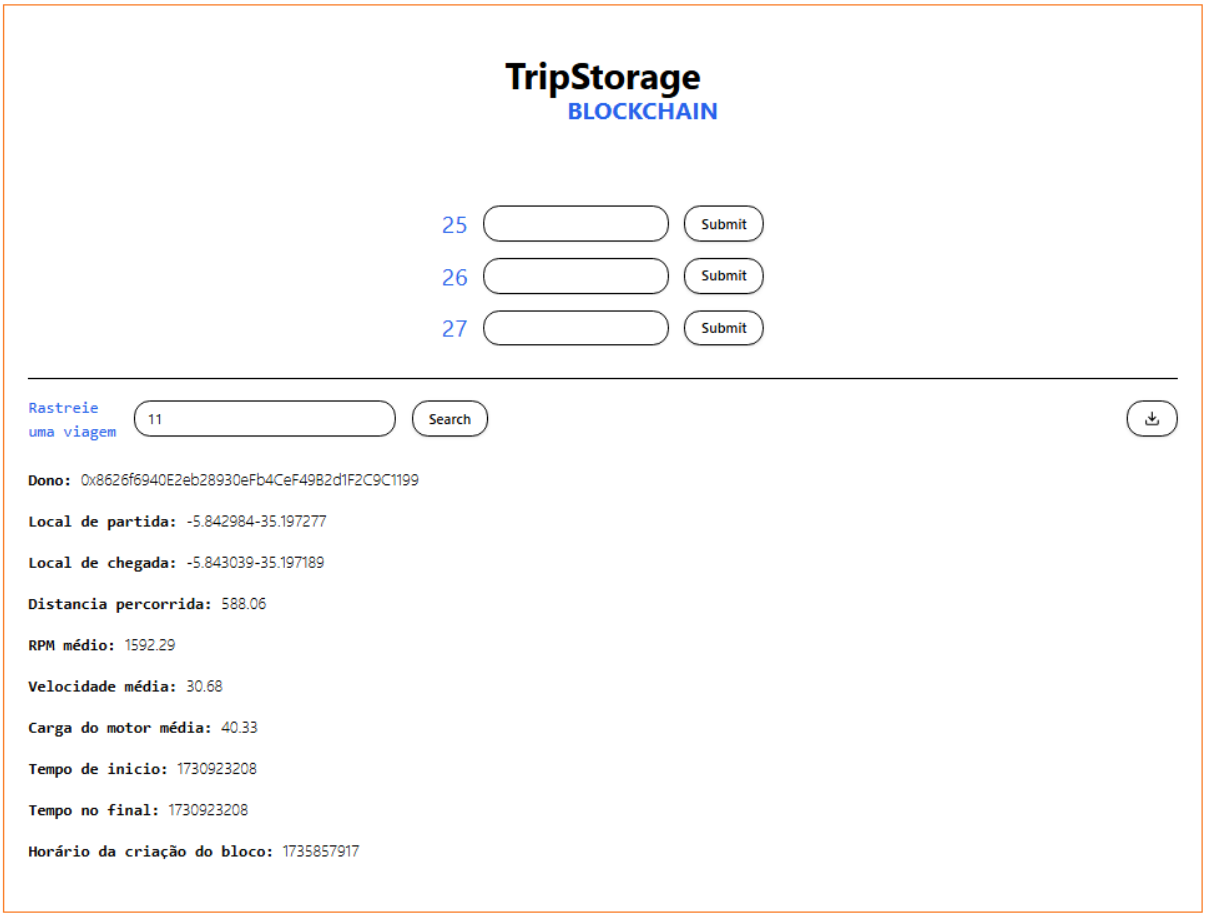
Fonte: Elaborada pelo autor, 2025.

Por fim, para atestar tudo que foi proposto, a funcionalidade de rastreabilidade foi demonstrada. Utilizando o ID da viagem cadastrada na *blockchain*, foi possível acessar as informações relevantes do resumo da viagem diretamente da rede. A Figura 27, ilustra essa etapa, exibindo todas as informações cadastradas, buscadas a partir do ID da viagem na *blockchain*.

Pelo ilustrado na imagem é possível identificar informações referentes a viagem como o endereço do dono, as coordenadas do local de partida e de chegada, a distância percorrida, o RPM médio durante a viagem, a velocidade média, a carga do motor média, o tempo de início e o tempo de finalização e o horário de criação do bloco na rede.

Dessa forma, verifica-se que tanto a rede quanto o sistema propostos no presente estudo foram capazes de executar com sucesso os procedimentos idealizados. Assim, foi possível demonstrar a solução envolvendo uma *blockchain* privada, com objetivo de registrar dados veiculares, desde a coleta nos sensores do veículo até a consolidação final na rede *blockchain*, indicando a viabilidade da solução para utilização prática.

Figura 27 – Rastreamento das informações salvas na *blockchain*.



Fonte: Elaborada pelo autor, 2025.

5 CONCLUSÃO

A evolução tecnológica impulsionada pela Indústria 4.0, especialmente com os avanços em IoT e suas aplicações na IoV, tem transformado o setor automotivo, gerando uma demanda crescente por soluções que assegurem a confiabilidade, rastreabilidade e segurança dos dados gerados por veículos conectados. Inserido nesse contexto, este estudo propôs uma solução integrada para o armazenamento de dados veiculares em uma rede *blockchain* privada, conectando informações coletadas em sistemas de processamento na borda a um registro descentralizado e imutável. A arquitetura desenvolvida estabelece uma integração funcional e segura entre sensores, uma API intermediária, banco de dados, *blockchain* e uma interface de usuário.

Por meio da prova de conceito realizada demonstrou-se a viabilidade do sistema ao registrar uma viagem de veículo em uma rede *blockchain* permissiva. Desde a coleta dos dados pelo dispositivo Freematics One+, passando pelo envio dos dados à API e armazenamento local em um arquivo CSV, até o registro final dos dados na *blockchain*, todos os componentes da arquitetura operaram de forma coesa. O processo de validação incluiu a verificação da comunicação entre os elementos e a confirmação das transações na rede *blockchain*, evidenciada por registros exibidos na ferramenta de exploração de blocos. Além disso, a funcionalidade de rastrear viagens cadastradas na *blockchain* mostrou-se eficiente, permitindo acessar informações relevantes por meio do ID de cada viagem, mas estando no na base de registros imutáveis.

A *blockchain* desenvolvida com a ferramenta Hyperledger Besu, foi capaz de armazenar e gerenciar dados veiculares de forma segura e auditável, destacando o potencial da *blockchain* como uma ferramenta estratégica para resolver problemas relacionados à confiabilidade e rastreabilidade de dados. Além disso, a utilização de contratos inteligentes no sistema simplificou processos automatizados, contribuindo para uma maior eficiência e transparência.

No entanto, embora os resultados tenham sido satisfatórios, algumas limitações e oportunidades para melhoria foram identificadas. Por exemplo, a segurança do sistema pode ser reforçada, especialmente em relação ao gerenciamento de chaves privadas. Além disso, é primordial testar a solução em ambientes de produção para avaliar seu desempenho em condições reais, bem como validar sua eficiência em cenários com um maior fluxo de dados de entrada.

Como direções futuras, sugere-se explorar a criação de contratos inteligentes voltados à geração de créditos de carbono, ampliando o impacto prático do sistema. Outras melhorias incluem a implementação de mecanismos para registrar novos participantes na rede de

forma segura e escalável, além de avanços na eficiência do sistema para lidar com volumes crescentes de dados. Essas iniciativas podem não apenas aprimorar a solução desenvolvida, mas também posicioná-la como uma ferramenta ainda mais promissora para aplicações relacionadas à IoV e sustentabilidade.

Dessa forma, este trabalho não apenas confirmou a viabilidade da aplicação de tecnologias emergentes no contexto de veículos conectados. Através da solução, foi possível destacar o papel da *blockchain* como tecnologia fundamental para o futuro da IoV. Portanto, o trabalho também abriu novas perspectivas para aprimoramentos futuros, de maneira que com as melhorias e avanços sugeridos, espera-se que a solução desenvolvida possa evoluir para atender a um espectro ainda maior de aplicações, ampliando seu impacto tanto na academia quanto na indústria, proporcionando uma mobilidade inteligente e sustentável.

REFERÊNCIAS

ARTHURS, P. et al. A taxonomy and survey of edge cloud computing for intelligent transportation systems and connected vehicles. *IEEE Transactions on Intelligent Transportation Systems*, v. 23, n. 7, p. 6206–6221, 2022.

BANKS, A.; PORCELLO, E. *Learning React: Modern Patterns for Developing React Apps.* [S.l.]: O'Reilly, 2020.

BASHIR, I. *Mastering Blockchain: Inner workings of blockchain, from cryptography and decentralized identities, to DeFi, NFTs and Web3.* [S.l.]: Packt, 2023.

BESU. *Hyperledger Besu for private networks.* Disponivel em: <https://besu.hyperledger.org/24.5.0/private-networks>, 2024.

BESU. *Run Tessera with Besu.* Disponivel em: <https://besu.hyperledger.org/24.6.0/private-networks/how-to/use-privacy/tessera>, 2024.

BUTERIN, V. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.* Disponivel em: <https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf>, 2014.

CAPOCASALE, V.; GOTTA, D.; PERBOLI, G. Comparative analysis of permissioned blockchain frameworks for industrial applications. *Blockchain: Research and Applications*, v. 4, n. 1, p. 100113, 2023. ISSN 2096-7209. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2096720922000549>https://www.sciencedirect.com/science/article/pii/S2096720922000549.

DOCKER. *Docker Compose.* Disponivel em: <https://docs.docker.com/compose/>, 2024.

DWIVEDI, S. K. et al. Blockchain-based secured event-information sharing protocol in internet of vehicles for smart cities. *Computers Electrical Engineering*, v. 86, p. 106719, 2020. ISSN 0045-7906. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0045790620305747>https://www.sciencedirect.com/science/article/pii/S0045790620305747.

FARAJI, M.; AYACHE, M.; BAKKALI, H. E. Security challenges in integrating biot systems: A comprehensive review. In: *2024 7th International Conference on Advanced Communication Technologies and Networking (CommNet).* [S.l.: s.n.], 2024. p. 1–6.

FERDOUS, M. S. et al. Immutable autobiography of smart cars leveraging blockchain technology. *The Knowledge Engineering Review*, v. 35, p. e3, 2020.

FERRARI, L.; PIROZZI, E. *Learn PostgreSQL: Use, manage, and build secure and scalable databases with PostgreSQL 16.* [S.l.]: Packt, 2023.

Ferreira X., G. et al. Fleet profile: Using visual analytics to prospect logistic solutions in industrial vehicles fleet. *Computers in Industry*, v. 151, p. 103971, 2023. ISSN 0166-3615. Disponível em: <https://www.sciencedirect.com/science/article/pii-/S0166361523001215>https://www.sciencedirect.com/science/article/pii/S0166361523001215.

GARG, R. *Blockchain for Real World Applications.* [S.l.]: Wiley, 2023.

GOUGH, J. et al. *Mastering API Architecture: Design, Operate, and Evolve API-Based Systems.* [S.l.]: O'Reilly, 2022.

HAMMOUD, A. et al. Ai, blockchain, and vehicular edge computing for smart and secure iov: Challenges and directions. *IEEE Internet of Things Magazine*, v. 3, n. 2, p. 68–73, 2020.

KANE, P. S.; MATTHIAS, K. *Docker: Up Running: Shipping Reliable Containers in Production.* [S.l.]: O'Reilly, 2023.

KULKARNI, K. *Learn Bitcoin and Blockchain: Understanding blockchain and Bitcoin architecture to build decentralized applications.* [S.l.]: Packt, 2018.

LAURENCE, T. *Blockchain for dummies.* [S.l.]: John Wiley Sons, 2019.

LEE, C. et al. Blockchain explorer based on rpc-based monitoring system. In: *2019 IEEE International Conference on Blockchain and Cryptocurrency (ICBC).* [S.l.: s.n.], 2019. p. 117–119.

LI, K. et al. As strong as its weakest link: How to break blockchain dapps at rpc service. In: *28th Annual Network and Distributed System Security Symposium, NDSS 2021.* [S.l.]: The Internet Society, 2021.

MODI, R. *Solidity Programming Essentials.* [S.l.]: Packt, 2022.

MOLLAH, M. B. et al. Blockchain for the internet of vehicles towards intelligent transportation systems: A survey. *IEEE Internet of Things Journal*, v. 8, n. 6, p. 4157–4185, 2021.

NAKAMOTO, S. *Bitcoin: A Peer-to-Peer Electronic Cash System.* Disponivel em: <https://bitcoin.org/bitcoin.pdf>, 2008.

NICKOLOFF, J.; KUENZLI, S. *Docker in Action.* [S.l.]: Manning, 2019.

OKERE, E. E.; BALYAN, V. Advances in blockchain-based internet of vehicles application: Prospect for machine learning integration. *Future Internet*, v. 16, n. 12, 2024. ISSN 1999-5903. Disponível em: <https://www.mdpi.com/1999-5903/16/12-/449>https://www.mdpi.com/1999-5903/16/12/449.

PANDA, S. S. et al. Distributed ledger technology for securing iot. In: *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT).* [S.l.: s.n.], 2020. p. 1–6.

PHILIP, B. V. et al. Distributed real-time iot for autonomous vehicles. *IEEE Transactions on Industrial Informatics*, v. 15, n. 2, p. 1131–1140, 2019.

QUORUM. *Configure QBFT consensus*. Disponivel em: <https://docs.goquorum.consensys.io/configure-and-manage/configure/consensus-protocols/qbft>, 2024.

SCHEID, E. J. et al. Bifröst: a modular blockchain interoperability api. In: *2019 IEEE 44th Conference on Local Computer Networks (LCN)*. [S.l.: s.n.], 2019. p. 332–339.

SIGNORETTI, G. et al. Performance evaluation of an evolving data compression algorithm embedded into an obd-ii edge device. In: *2020 IEEE International Workshop on Metrology for Industry 4.0 IoT*. [S.l.: s.n.], 2020. p. 696–701.

SPIGOLON, M.; SINIK, M.; COLLINA, M. *Accelerating Server-Side Development with Fastify: A comprehensive guide to API development for building a scalable backend for your web apps*. [S.l.]: Packt, 2023.

VITE. *Why Vite*. Disponivel em: <https://vite.dev/guide/why.html>, 2024.

WANG, D.; ZHANG, X. Secure data sharing and customized services for intelligent transportation based on a consortium blockchain. *IEEE Access*, v. 8, p. 56045–56059, 2020.

WOOD, G. *Ethereum: a secure decentralised generalised transaction ledger*. Disponivel em: <https://ethereum.github.io/yellowpaper/paper.pdf>, 2024.

WU, B. X.; ZOU, Z.; SONG, D. *Learn Ethereum: A practical guide to help developers set up and run decentralized applications with Ethereum 2.0*. [S.l.]: Packt, 2023.

ZAND, M.; WU, B. X.; MORRIS, A. M. *Hands-On Smart Contract Development with Hyperledger Fabric V2: Building Enterprise Blockchain Applications*. [S.l.]: O'Reilly, 2021.